Tài liệu chi tiết đề tài MLOps

Mục lục

1. Giới thiệu 3
1.1 Vấn đề, bài toán và giải pháp 3
1.2 Nghiệp vụ và dữ liệu 3
1.3 Bảng và cột dữ liệu 3
1.4 Ba bài toán và sáu model 5
1.5 Vì sao ba bài toán và sáu model 5
1.6 Kiến trúc tổng thể 6
1.7 Mục đích tài liệu 6
1.8 Phạm vi 6
1.9 Quy ước và nguồn thông tin 7
1.10 Thuật ngữ 7
2. Kiến trúc tổng quan 8
2.1 Sơ đồ kiến trúc 8
2.2 Các thành phần 8
2.3 Lịch vận hành 9
2.4 Điều phối bằng Airflow 10
2.5 Nơi lưu dữ liệu 11
2.6 Chuẩn bị môi trường và truy cập 12
3. Luồng 1: Gold → Feature Store 14
3.1 Mục tiêu và nguyên tắc 14
3.2 Sơ đồ luồng 14
3.3 Cách lưu: partition theo report_date 15
3.4 Từng bước hằng ngày 16
3.5 Cách kiểm tra 17
4. Luồng 2: Huấn luyện và đưa model lên production 18
4.1 Tổng quan 18
4.2 Dữ liệu huấn luyện và cách chia thời gian 18
4.3 Từng bước 19
4.4 Áp dụng cho overdue_risk và campaign_response 24
4.5 Kết quả và dữ liệu sinh ra 25
4.6 Thế nào là model tốt hơn 25
4.7 Chất lượng model 26
5. Luồng 3: Dự đoán batch và giám sát drift 27
5.1 Tổng quan 27
5.2 Dự đoán: predict 28
5.3 Giám sát drift: drift 29
5.4 Quyết định huấn luyện lại: retrain_check 31
5.5 Áp dụng cho overdue_risk và campaign_response 32
6. Luồng 4: Triển khai model lên KServe 33
6.1 Tổng quan 33
6.2 Vai trò của KServe: phục vụ ai, để làm gì 33
6.3 Từng bước 34
7. Luồng 5: Đánh giá thực tế và huấn luyện lại 37
7.1 Tổng quan 37
7.2 Từng bước 37
7.3 Vòng huấn luyện lại 39
8. Tổng kết và lệnh thường dùng 40
8.1 Tổng kết 40
8.2 Lệnh thường dùng 40
8.3 Hạn chế và hướng phát triển 41

1. Giới thiệu

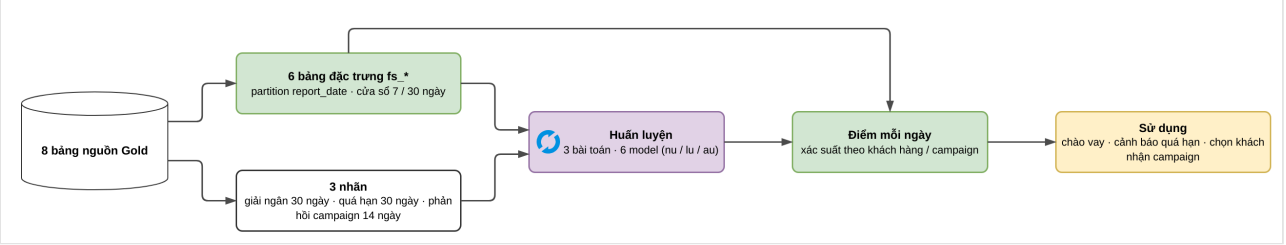
1.1 Vấn đề, bài toán và giải pháp

Bảng 1.1: Vấn đề, bài toán, giải pháp

	Nội dung
Vấn đề	Công ty chứng khoán cho khách hàng vay ký quỹ (margin) để mua cổ phiếu. Bộ phận kinh doanh và chăm sóc khách hàng không thể liên hệ toàn bộ khách hàng mỗi ngày, trong khi chỉ một phần nhỏ khách hàng thực sự có nhu cầu vay trong thời gian tới. Chọn khách hàng thủ công thì chậm, không nhất quán và khó đo hiệu quả.
Bài toán	Ba bài toán phân loại nhị phân trên cùng nguồn dữ liệu: (1) **loan_propensity**: khách hàng có giải ngân margin mới trong 30 ngày tới không; (2) **overdue_risk**: khách hàng đang có dư nợ có bắt đầu quá hạn trong 30 ngày tới không; (3) **campaign_response**: sau khi nhận một chiến dịch ưu đãi, khách hàng có giải ngân trong 14 ngày không. Tổng cộng 6 model (mục 1.4).
Giải pháp	Một hệ thống MLOps tự động từ dữ liệu đến dịch vụ: đặc trưng tính theo ngày, huấn luyện có kiểm soát phiên bản, chấm điểm hằng ngày, giám sát drift và hiệu quả thực tế, triển khai model qua GitOps. Kết quả là bảng điểm mỗi ngày để chào vay đúng người, cảnh báo sớm rủi ro quá hạn và chọn khách hàng nhận chiến dịch.

1.2 Nghiệp vụ và dữ liệu

Dữ liệu đầu vào là 8 bảng nguồn ở tầng Gold: `customer_account`, `stock_transaction`, `margin_balance_daily`, `margin_cash_flow`, `stock_price_daily`, `security_master`, `portfolio_holding_daily`, `customer_campaign`. Từ đó tính 6 nhóm đặc trưng theo ngày (nhân khẩu, hành vi giao dịch, số dư margin, tài sản, danh mục, chiến dịch). Đặc trưng dùng cửa sổ theo ngày lịch 7 và 30 ngày. Nhãn của cả ba bài toán đều nhìn về tương lai sau ngày chấm (30 hoặc 14 ngày) và được tính từ dòng tiền margin, trạng thái margin và lịch sử chiến dịch; nhãn không bao giờ nằm trong bảng đặc trưng.



Hình 1.1: Dòng dữ liệu tổng quát: từ bảng nguồn đến điểm dự đoán

1.3 Bảng và cột dữ liệu

Tầng Gold (nguồn, hệ thống chỉ đọc). Tám bảng, mỗi bảng một vai trò:

Bảng 1.2: Các bảng Gold nguồn

Bảng Gold	Khoá / đơn vị dòng	Cột chính	Dùng cho
`customer_account`	`master_account`	account_open_date, account_status (ACTIVE / DORMANT / CLOSED), account_close_date, dormant_start_date, customer_group, risk_profile	Điều kiện tài khoản hợp lệ, tuổi tài khoản, hệ số tài sản đảm bảo

`stock_transaction`	`transaction_id`	master_account, symbol, transaction_time, side (BUY / SELL), quantity, price, trade_value	Hành vi giao dịch 7 / 30 ngày
`margin_balance_daily`	(master_account, business_date)	margin_debt, margin_limit, asset_value, cash_balance, collateral_value, NAV, margin_status (NO_MARGIN / ACTIVE / OVERDUE)	Số dư margin, tài sản, nhãn quá hạn
`margin_cash_flow`	`cash_flow_id`	master_account, event_date, fk_cash_flow_type, fk_deliver_type, amount, status (POSTED / CANCELLED)	Phân segment nu / lu / au, nhãn giải ngân
`stock_price_daily`	(symbol, ngày)	Giá thị trường theo ngày của từng mã	Giá trị danh mục, tỷ trọng lỗ
`security_master`	`symbol`	Ngành, thanh khoản và mức rủi ro của mã	Tỷ trọng tài sản rủi ro
`portfolio_holding_daily`	(master_account, ngày, symbol)	Số lượng nắm giữ theo mã mỗi ngày	Danh mục, mức tập trung
`customer_campaign`	`campaign_event_id`	campaign_id, master_account, sent_at, channel (APP / SMS / EMAIL), offer_type, interest_rate_offer, status (DELIVERED / FAILED)	Đặc trưng chiến dịch; nhãn campaign_response (không chứa kết quả phản hồi)

Feature Store: sáu bảng `fs_*`, khoá chung (master_account, report_date), partition theo `report_date`, mỗi ngày một dòng cho mỗi tài khoản ACTIVE đã mở từ 30 ngày. Tổng cộng 24 đặc trưng cho model:

Bảng 1.3: Sáu bảng đặc trưng (24 đặc trưng model)

Bảng fs_*	Cột đặc trưng	Ý nghĩa
`fs_demographic`	account_age_days; (margin_segment)	Tuổi tài khoản; segment nu / lu / au để chia model
`fs_trading_behavior`	trade_count_7d, trade_count_30d, total_trading_amount_7d, total_trading_amount_30d, active_trading_days_30d	Tần suất và giá trị giao dịch gần đây
`fs_margin_balance`	margin_debt, margin_limit, margin_utilization_ratio, debt_change_7d, over_limit_streak, over_limit_days_30d	Mức vay, mức dùng hạn mức, chuỗi ngày vượt hạn mức
`fs_asset_balance`	cash_balance, asset_value, collateral_value, cash_to_asset_ratio, NAV	Tiền, tài sản, tài sản đảm bảo, tài sản ròng
`fs_portfolio`	holding_symbol_count, portfolio_concentration, unrealized_loss_ratio, risky_asset_ratio	Cấu trúc và rủi ro danh mục
`fs_campaign`	campaign_count_30d, days_since_last_campaign, has_campaign_history	Lịch sử nhận chiến dịch

Bộ dữ liệu huấn luyện được dựng từ fs_* và nhãn tính từ Gold, không nằm trong fs_*:

Bảng 1.4: Bộ dữ liệu huấn luyện

Bộ dữ liệu	Dòng	Cột ngoài đặc trưng
`training_dataset`	Khách hàng × ngày	master_account, report_date, `margin_segment`, `overdue_eligible`, nhãn `loan_label_next`, `overdue_label_next_30d`, `split` (train / validation / test)
`campaign_training_dataset`	Một lần gửi chiến dịch	campaign_event_id, campaign_id, channel, offer_type, interest_rate_offer, margin_segment, nhãn `campaign_response_label_14d`, `split`, kèm các đặc trưng tài khoản tại ngày gửi

1.4 Ba bài toán và sáu model

Bảng 1.5: Ba bài toán và sáu model

Bài toán	Câu hỏi	Nhãn	Đơn vị dữ liệu	Model
loan_propensity	Khách hàng có giải ngân margin mới (POSTED) trong D+1…D+30?	`loan_label_next`	Khách hàng × ngày	3 model: `loan_propensity_nu`, `_lu`, `_au`
overdue_risk	Khách hàng có dư nợ hoặc lịch sử margin, chưa quá hạn tại D, có **bắt đầu** quá hạn trong D+1…D+30?	`overdue_label_next_30d`	Khách hàng × ngày, chỉ dòng `overdue_eligible = 1`	2 model: `overdue_risk_lu`, `_au` (nhóm nu chưa có nợ)
campaign_response	Chiến dịch gửi thành công (DELIVERED) ngày D có dẫn tới giải ngân trong D+1…D+14?	`campaign_response_label_14d`	Sự kiện gửi chiến dịch	1 model: `campaign_response_all` (segment là một đặc trưng)

Cả ba bài toán dùng chung nguồn Gold, chung 6 bảng fs_* và chung hạ tầng. Mỗi bài toán có cấu hình riêng trong `training/usecases.yaml` (nhãn, danh sách đặc trưng cho phép, segment, trọng số lớp, quy tắc ngưỡng) và bộ DAG riêng cùng cấu trúc: `<usecase>_train`, `<usecase>_predict`, `<usecase>_evaluate`. Các chương 3–7 mô tả luồng chung và lấy **loan_propensity** làm ví dụ chi tiết (ảnh chụp, số liệu); điểm khác của hai bài toán còn lại được nêu ở cuối mỗi chương.

1.5 Vì sao ba bài toán và sáu model

Đề tài nhằm chứng minh một **nền tảng MLOps dùng chung** chứ không phải một model đơn lẻ. Ba bài toán được chọn để mỗi bài toán kiểm chứng một khía cạnh khác của nền tảng, cùng chạy trên một nguồn dữ liệu và một hạ tầng:

Bảng 1.6: Mỗi bài toán kiểm chứng điều gì

Bài toán	Dạng bài toán	Số model	Điều nó kiểm chứng
loan_propensity	Một model cho mỗi segment (nu / lu / au) trên đơn vị khách hàng × ngày	3	Quản lý nhiều model của cùng một bài toán: mỗi model một ngưỡng và một AUC riêng, cổng champion / challenger all-or-nothing trên cả ba segment

overdue_risk	Chỉ chấm tập con đủ điều kiện (`overdue_eligible = 1`), nhãn là sự kiện bắt đầu quá hạn	2	Cùng khung huấn luyện nhưng đổi nhãn, đổi tập khách hàng được chấm và bỏ nhóm không có nợ
campaign_response	Đơn vị dữ liệu khác: mỗi dòng là một lần gửi chiến dịch, segment là một đặc trưng	1	Hệ thống xử lý được bộ dữ liệu khác đơn vị và đặc trưng phân loại, không chỉ dạng khách hàng × ngày

Vì sao loan_propensity tách thành ba model theo segment: ba nhóm khách hàng có hành vi và tỷ lệ nhãn dương khác nhau (khoảng 14% ở nu, 28% ở lu, 38% ở au), nên mỗi nhóm có ngưỡng cảnh báo và chất lượng riêng cần theo dõi riêng. Tổng cộng 3 + 2 + 1 = 6 model.

Bảng 1.7: Mục tiêu chứng minh của đề tài

Mục tiêu chứng minh	Bằng chứng trong hệ thống
Thêm bài toán mới không đổi hạ tầng	Cả ba bài toán dùng chung Gold, 6 bảng fs_*, image pipeline và mẫu DAG train / predict / evaluate; chỉ khác cấu hình trong `training/usecases.yaml` (nhãn, đặc trưng cho phép, segment, trọng số lớp, ngưỡng)
Vòng đời nhiều model chạy song song	Mỗi model có tên `<usecase>_<segment>` trong MLflow, version và alias staging / production độc lập
Mọi model qua cùng một cổng kiểm soát	Cổng champion / challenger, người vận hành duyệt, drift và đánh giá hằng tuần áp dụng giống nhau cho cả sáu model
Dùng được cho nhiều dạng dữ liệu	Dữ liệu theo khách hàng × ngày (hai bài toán) và theo sự kiện gửi chiến dịch (một bài toán)

1.6 Kiến trúc tổng thể

Hệ thống gồm bốn khối: (1) tính đặc trưng, (2) huấn luyện và quản lý model, (3) chấm điểm và giám sát hằng ngày, (4) triển khai model lên dịch vụ. Airflow điều phối các khối; MLflow giữ model và alias; MinIO và PostgreSQL lưu dữ liệu; ArgoCD và KServe đưa model lên dịch vụ. Sơ đồ chi tiết ở chương 2, từng luồng ở chương 3 đến 7.

> **Giới hạn phạm vi.** Tài liệu này thuộc phạm vi MLOps. Bảng Gold được coi là **luôn có dữ liệu mới mỗi ngày**; việc nạp, làm sạch và đưa dữ liệu vào Gold nằm ngoài phạm vi và không được mô tả.

1.7 Mục đích tài liệu

Tài liệu mô tả từng luồng của hệ thống MLOps cho ba bài toán margin (**loan_propensity**, **overdue_risk**, **campaign_response**): mỗi luồng chạy khi nào, Airflow gọi thành phần nào, thành phần đó làm gì, dữ liệu được lưu ở đâu và dùng lại như thế nào. Mỗi bước có giải thích và câu lệnh cụ thể để tự chạy, tự kiểm tra và tự chụp ảnh minh hoạ.

Tài liệu không đi vào từng dòng code. Người đọc cần hiểu được: *dữ liệu đi qua những bảng nào, được ghi theo cách nào (ghi đè hay thêm), mô hình nào được dùng cho ngày nào, và ai là người quyết định đưa mô hình lên production*.

1.8 Phạm vi

Bảng 1.8: Phạm vi tài liệu

Nội dung	Trong phạm vi	Ngoài phạm vi
Dữ liệu và đặc trưng	8 bảng nguồn Gold (Iceberg), 6 nhóm bảng fs_*, nhãn tính từ dòng tiền Gold	Feast, Redis (online store)

Huấn luyện	Huấn luyện theo segment au / lu / nu, ghi MLflow, cổng champion / challenger, người vận hành duyệt	—
Dự đoán	Batch hằng ngày, ghi bảng điểm Iceberg, giám sát drift	Dự đoán thời gian thực qua API gateway
Triển khai	Deployer commit manifest, ArgoCD đồng bộ, KServe phục vụ model @production	Dự đoán trực tuyến (online serving) qua API gateway và Feast/Redis: hướng phát triển (mục 6.2)
Giám sát	Drift (Evidently), đánh giá hiệu quả thực tế theo tuần	—

1.9 Quy ước và nguồn thông tin

- **Nguồn thông tin**: tên DAG, tham số, ngưỡng, bảng và đường dẫn trong tài liệu lấy từ mã nguồn của dự án. Số liệu minh hoạ lấy từ lần chạy ngày 26/10/2026 (version 11 của các model loan_propensity).
- **Giờ**: Airflow hiển thị giờ UTC. Giờ Việt Nam = UTC + 7. Ví dụ `@daily` = 00:00 UTC = 07:00 giờ Việt Nam.
- **Lệnh**: chạy tại thư mục gốc repo, dùng `uv run` cho Python và `kubectl` / `helm` cho cụm Kubernetes.
- **Ảnh chụp** chỉ mang tính tham khảo; giá trị thực tế (ngày, version, số dòng) sẽ khác khi chạy lại.

1.10 Thuật ngữ

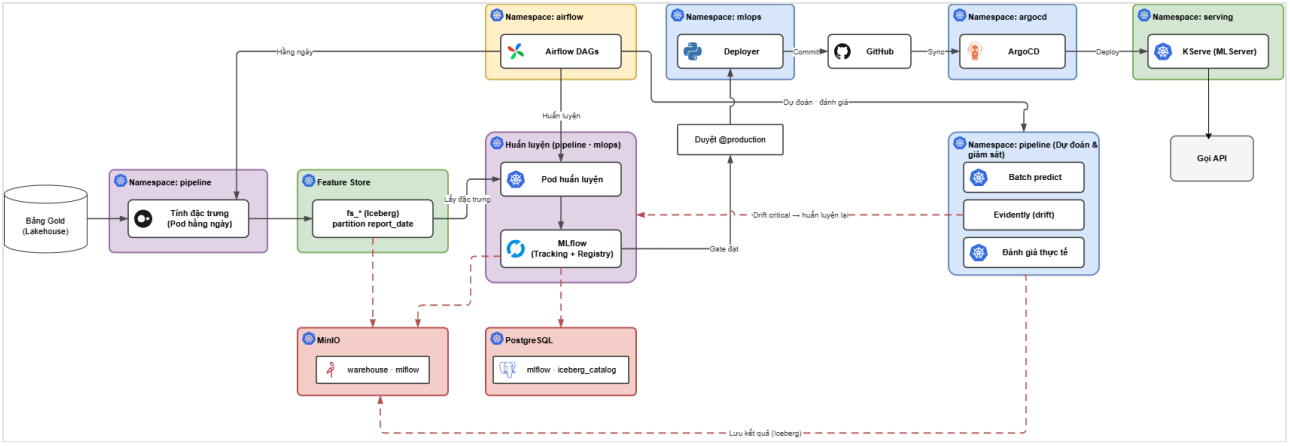
Bảng 1.9: Thuật ngữ

Thuật ngữ	Ý nghĩa
Gold	Các bảng nguồn đã làm sạch (khách hàng, giao dịch, margin, giá, danh mục…), nguồn duy nhất để tính đặc trưng và nhãn.
fs_* (Feature Store)	6 bảng đặc trưng: demographic, trading_behavior, margin_balance, asset_balance, portfolio, campaign. Khoá (master_account, report_date).
report_date	Ngày dữ liệu mà đặc trưng mô tả; là cột partition của bảng fs_*.
Use case (bài toán)	Một bài toán dự đoán với nhãn và cấu hình riêng: loan_propensity, overdue_risk, campaign_response.
Segment	Nhóm khách hàng theo margin: **nu**, **lu**, **au**. Bài toán theo segment có một model cho mỗi segment.
Nhãn loan_label_next	1 nếu khách hàng có giải ngân mới được ghi nhận (POSTED) trong 30 ngày sau ngày chấm (D+1…D+30), ngược lại 0.
Alias staging / production	Tên trỏ vào một version của model đã đăng ký trong MLflow. staging = vừa huấn luyện xong; production = đang được dùng.
Champion / challenger	Champion là version đang @production; challenger là version mới ở @staging.
Drift	Phân phối đặc trưng ngày hiện tại lệch so với ngày tham chiếu.
Deployer	Tiến trình đọc alias @production và commit manifest KServe lên GitHub.

2. Kiến trúc tổng quan

2.1 Sơ đồ kiến trúc

Toàn bộ hệ thống chạy trên một cụm Kubernetes, tách theo namespace: `airflow` (điều phối), `pipeline` (pod xử lý), `mlops` (MLflow, MinIO, PostgreSQL, deployer), `argocd` (GitOps) và `serving` (KServe). Mỗi bước tính đặc trưng, huấn luyện, dự đoán, drift và đánh giá là một pod riêng do Airflow tạo, chạy xong thì kết thúc.



Hình 2.1: Kiến trúc tổng quan

2.2 Các thành phần

Bảng 2.1: Thành phần hệ thống

Thành phần	Chạy ở đâu	Vai trò
Job tính đặc trưng	Kubernetes, namespace pipeline	Đọc Gold, tính 6 nhóm fs_*, kiểm tra chất lượng, tạo run đặc trưng có phiên bản.
Airflow 2.10.5	Kubernetes, namespace airflow	Lập lịch DAG `feature_store_daily` và bộ DAG train / predict / evaluate của từng bài toán. Dùng KubernetesPodOperator tạo pod ở namespace `pipeline`.
Pod mlops-pipeline	Kubernetes, namespace pipeline	Image `mlops-pipeline`: huấn luyện, cổng champion, dự đoán, drift, đánh giá. Đọc run đặc trưng từ PVC `pipeline-data`.
MLflow 3.15.2	Kubernetes, namespace mlops	Ghi run, đăng ký model, giữ alias staging / production.
MinIO	Kubernetes, namespace mlops	Hai bucket: `mlflow` (file model, artifact) và `warehouse` (bảng pred_, drift_, eval_ và báo cáo drift).
PostgreSQL	Kubernetes, namespace mlops	Hai cơ sở dữ liệu: `mlflow` (metadata) và `iceberg_catalog` (catalog của các bảng giám sát).
Deployer	Kubernetes, namespace mlops	Đọc @production, commit manifest lên GitHub (nhánh master, thư mục serving/k8s).
ArgoCD 3.5.5	Kubernetes, namespace argocd	Application `serving`: tự đồng bộ Git vào cụm (auto-sync, prune, self-heal).
KServe 0.21.0	Kubernetes, namespaces kserve và serving	InferenceService chạy model bằng MLServer 1.7.1 (runtime `mlflow-mlserver`), kiểu RawDeployment.

2.3 Lịch vận hành

Lịch vận hành trong một tuần (giờ Việt Nam; Airflow hiển thị UTC, trừ 7 giờ)

	Thứ Hai	Thứ Ba	Thứ Tư	Thứ Năm	Thứ Sáu	Thứ Bảy	Chủ Nhật
DAG Feature Store	06:00	06:00	06:00	06:00	06:00	06:00	06:00
DAG Dự đoán (@daily)	07:00	07:00	07:00	07:00	07:00	07:00	07:00
DAG Huấn luyện (@weekly)							07:00
DAG Đánh giá (@weekly)							07:00
Deployer (liên tục)	60 s	60 s	60 s	60 s	60 s	60 s	60 s

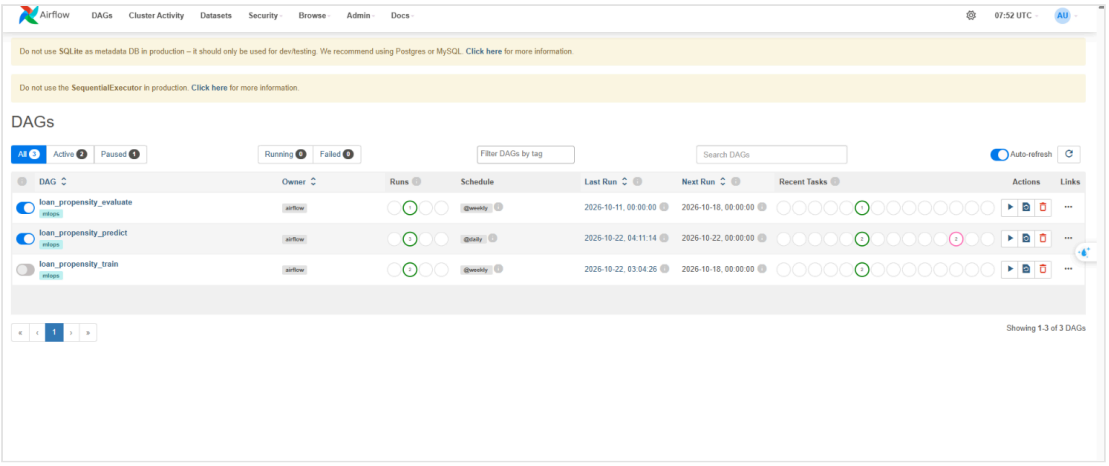
DAG Dự đoán còn gọi DAG Huấn luyện khi drift critical (trigger_retrain). Huấn luyện và Đánh giá cùng chạy Chủ Nhật 07:00.

Hình 2.2: Lịch vận hành trong tuần

Bảng 2.2: Lịch chạy

Việc	Khi nào (giờ VN)	Cơ chế
Tính đặc trưng	06:00 hằng ngày (23:00 UTC ngày trước)	DAG `feature_store_daily`; xong mới đánh dấu ngày hoàn tất
Dự đoán + drift	07:00 hằng ngày (`@daily`)	DAG `loan_propensity_predict`
Huấn luyện + cổng	Chủ Nhật 07:00 (`@weekly`) hoặc khi drift critical	DAG `loan_propensity_train`
Đánh giá thực tế	Chủ Nhật 07:00 (`@weekly`)	DAG `loan_propensity_evaluate`
Deployer	Liên tục: đọc alias mỗi 60 giây, kiểm tra cụm mỗi 600 giây	Deployment `deployer` chạy thường trực

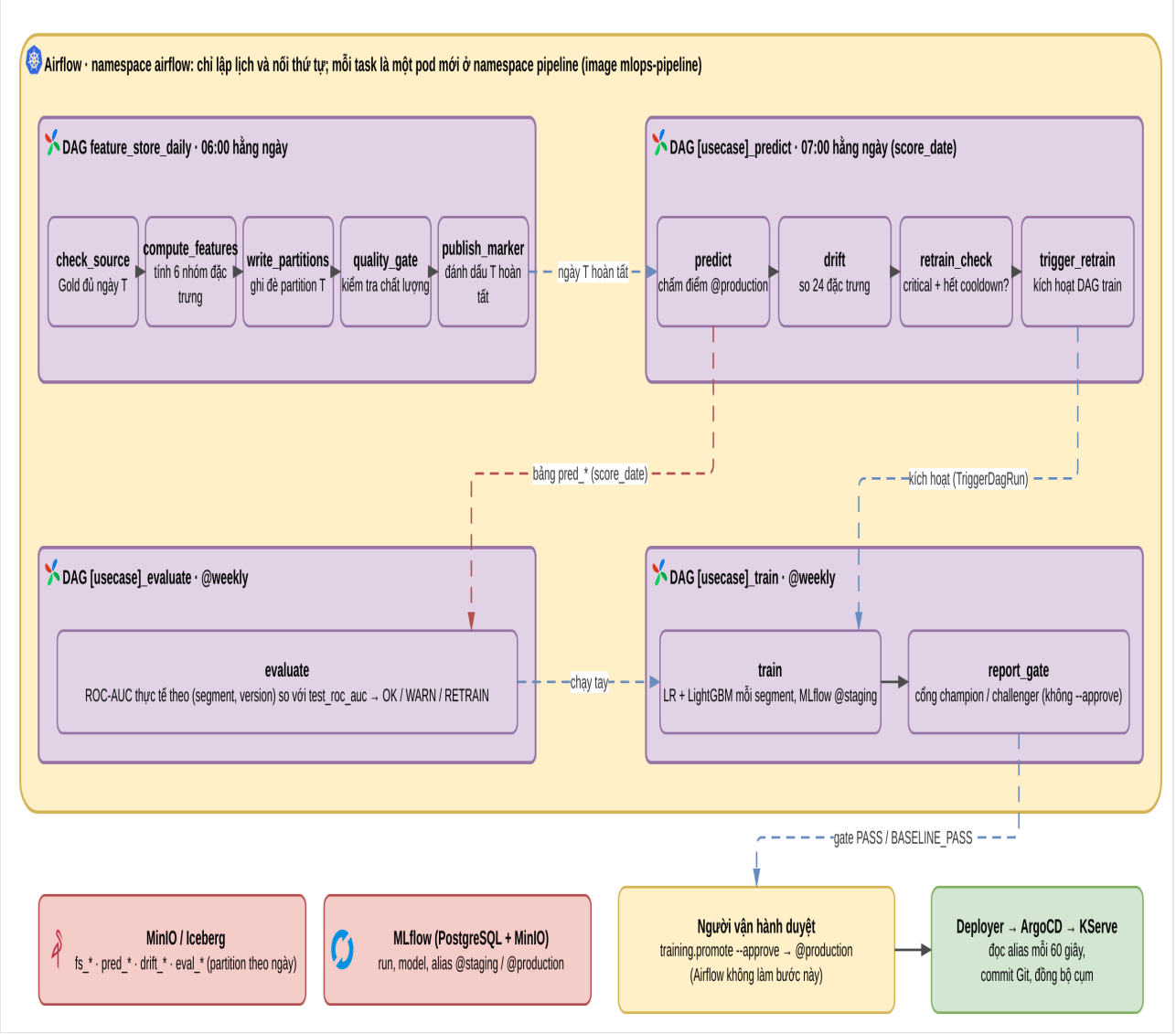
Vì sao ngày chạy là tham số DAG? Dữ liệu dùng trong tài liệu có mốc kết thúc cố định thay vì "hôm nay": đặc trưng kết thúc 30/01/2026 và nhãn kết thúc 31/12/2025. Vì vậy DAG dự đoán nhận `score_date` (mặc định 2026-01-30) và DAG đánh giá nhận `start` / `end` (mặc định 2025-12-25 và 2025-12-31). Cả ba DAG đặt `catchup=False`, `max_active_runs=1`.



Hình 2.3: Airflow: các DAG của bài toán loan_propensity với lịch @weekly / @daily

2.4 Điều phối bằng Airflow

Airflow chỉ **lập lịch và nối thứ tự**: mỗi task tạo một pod ở namespace `pipeline` chạy một lệnh Python rồi kết thúc. Có bốn nhóm DAG; ba nhóm sau lặp lại cho từng bài toán (`loan_propensity`, `overdue_risk`, `campaign_response`).



Hình 2.4: Tổng quan điều phối: các DAG, thứ tự task và cách các DAG liên kết với nhau

Bảng 2.3: Các nhóm DAG

DAG	Lịch	Task nối tiếp	Đọc	Ghi / kết quả
`feature_store_daily`	06:00 hằng ngày	check_source → compute_features → write_partitions → quality_gate → publish_marker	Gold	6 bảng fs_*; đánh dấu ngày T hoàn tất
`<usecase>_predict`	07:00 hằng ngày	predict → drift → retrain_check → trigger_retrain	fs_* ngày `score_date`, model @production	`pred_*`, `drift_*`, báo cáo HTML; có thể kích hoạt DAG train

`<usecase>_train`	@weekly (Chủ Nhật 07:00) hoặc được kích hoạt	train → report_gate	Run đặc trưng đã duyệt	Version mới @staging trong MLflow, kết quả cổng
`<usecase>_evaluate`	@weekly (Chủ Nhật 07:00)	evaluate	`pred_*` và nhãn đã chín	`eval_*`: OK / WARN / RETRAIN

Các DAG **không gọi trực tiếp nhau** mà liên kết theo ba cách:

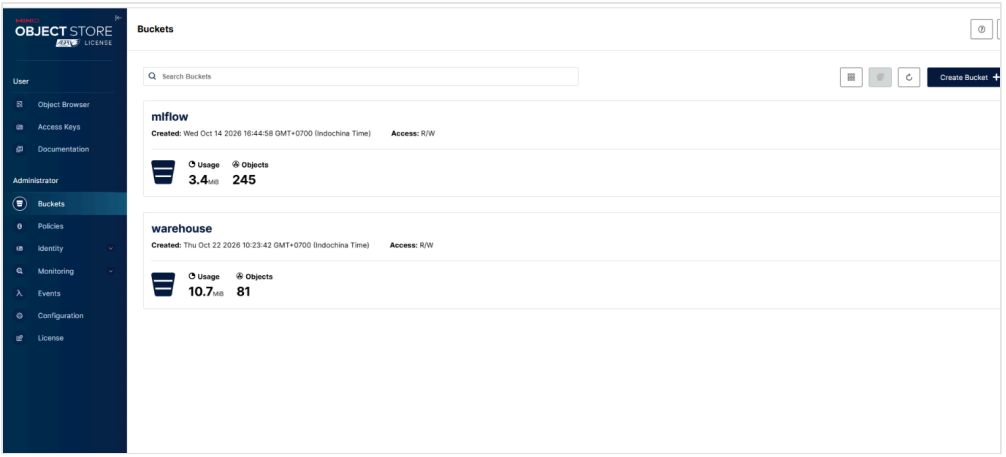
- **Theo thời gian**: đặc trưng tính lúc 06:00, dự đoán chạy lúc 07:00 và chỉ dùng ngày đã được đánh dấu hoàn tất.
- **Kích hoạt DAG**: task `trigger_retrain` của DAG dự đoán kích hoạt DAG huấn luyện khi drift ở mức critical, đã có nhãn mới hơn `data_end` của version production và đã hết cooldown.
- **Qua dữ liệu và alias**: DAG đánh giá đọc bảng `pred_*` do DAG dự đoán ghi; deployer đọc alias @production trong MLflow. Khi đánh giá báo WARN kéo dài hoặc RETRAIN, người vận hành chạy tay DAG huấn luyện.

> **Airflow không bao giờ đặt @production.** Huấn luyện luôn dừng ở @staging và `report_gate` chỉ in kết quả cổng (không dùng `--approve`). Chỉ người vận hành, bằng `training.promote --approve` hoặc MLflow UI, mới đặt alias production; việc triển khai sau đó do deployer và ArgoCD đảm nhận.

2.5 Nơi lưu dữ liệu

Bảng 2.4: Nơi lưu dữ liệu

Dữ liệu	Nơi lưu	Cách ghi
8 bảng Gold	Kho dữ liệu Gold (Iceberg)	Có dữ liệu mới mỗi ngày; hệ thống chỉ đọc
6 bảng fs_*	Iceberg trên MinIO (`s3://warehouse`), partition `report_date`	Mỗi ngày ghi đè đúng 1 partition
Run đặc trưng đã duyệt (training_dataset, bảng nhãn loan_overdue_labels, class_weights, quality_report)	PVC `pipeline-data`, thư mục `day4_runs/<RUN_ID>`	Mỗi lần chạy một thư mục mới; con trỏ `current.json`
Model, artifact huấn luyện	MinIO bucket `mlflow`; metadata trong PostgreSQL	Mỗi lần huấn luyện thêm version mới
Điểm dự đoán	Iceberg `monitoring.pred_loan_propensity`, partition `score_date`	Ghi đè partition của ngày chấm
Kết quả drift	Iceberg `monitoring.drift_loan_propensity` (partition `current_date`) + HTML ở `s3://warehouse/reports/drift/…`	Ghi đè theo ngày
Đánh giá thực tế	Iceberg `monitoring.eval_loan_propensity`, partition `window_end`	Thay kết quả của đúng cửa sổ (start, end)
Manifest triển khai	GitHub: `serving/k8s/30-isvc-<model>.yaml`	Mỗi lần đổi version là một commit



Hình 2.5: MinIO: hai bucket mlflow (model, artifact) và warehouse (bảng giám sát, báo cáo)

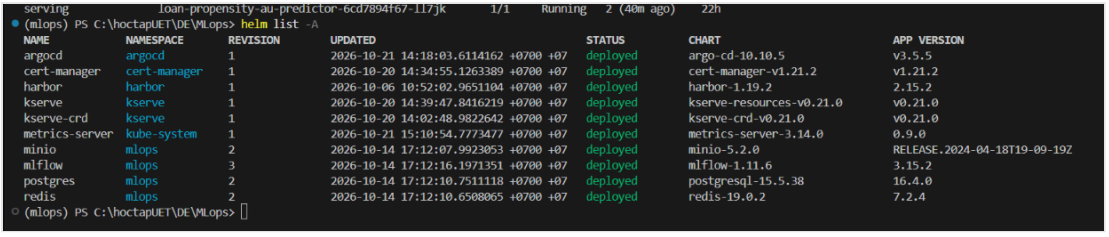
2.6 Chuẩn bị môi trường và truy cập

Các bước dưới đây chỉ làm một lần (hoặc khi khởi động lại máy). Các luồng sau giả định môi trường đã chạy.

1 Cài các thành phần nền vào cụm Kubernetes

Dùng Helmfile cài PostgreSQL, MinIO, MLflow, ArgoCD, metrics-server (release khai báo trong `infra/helmfile.yaml`); KServe và cert-manager cài bằng Helm. Kiểm tra mọi pod ở trạng thái Running.

```
helmfile -f infra/helmfile.yaml sync
helm list -A
kubectl get pods -A
```



Hình 2.6: Các release Helm trong cụm (argocd, cert-manager, kserve, minio, mlflow, postgres...)

2 Kết nối ArgoCD với repo và tạo Application serving

Application `serving` theo dõi thư mục `serving/k8s` trên nhánh master. Cần secret để ArgoCD đọc repo và secret MinIO để KServe tải model.

```
kubectl apply -f argocd/serving-app.yaml
kubectl -n argocd get applications
```

3 Build và đẩy image pipeline lên registry

Mọi pod xử lý dùng chung image `mlops-pipeline` (code huấn luyện, dự đoán, drift, đánh giá). Image gắn tag theo mã commit, nên mỗi model trong MLflow (tag `git_commit`) truy được về đúng phiên bản code đã sinh ra nó.

```
docker build -f pipeline/Dockerfile -t <registry>/mlops/mlops-pipeline:$(git rev-parse --short
HEAD) .
docker push <registry>/mlops/mlops-pipeline:$(git rev-parse --short HEAD)
```

4 Kiểm tra Airflow

Airflow chạy trong namespace `airflow`, dùng KubernetesPodOperator tạo pod ở namespace `pipeline`. Đăng nhập UI bằng tài khoản quản trị để xem DAG.

```
kubectl -n airflow get pods
kubectl -n airflow exec deploy/airflow -- airflow dags list
```

5 Truy cập các giao diện

Khi làm việc từ máy cá nhân, mở cổng tới các dịch vụ trong cụm bằng port-forward (hoặc qua ingress nội bộ).

```
kubectl -n airflow port-forward svc/airflow-webserver 8080:8080
kubectl -n mlops port-forward svc/mlflow 5050:80
kubectl -n mlops port-forward svc/minio-console 9001:9001
kubectl -n argocd port-forward svc/argocd-server 8090:80
```

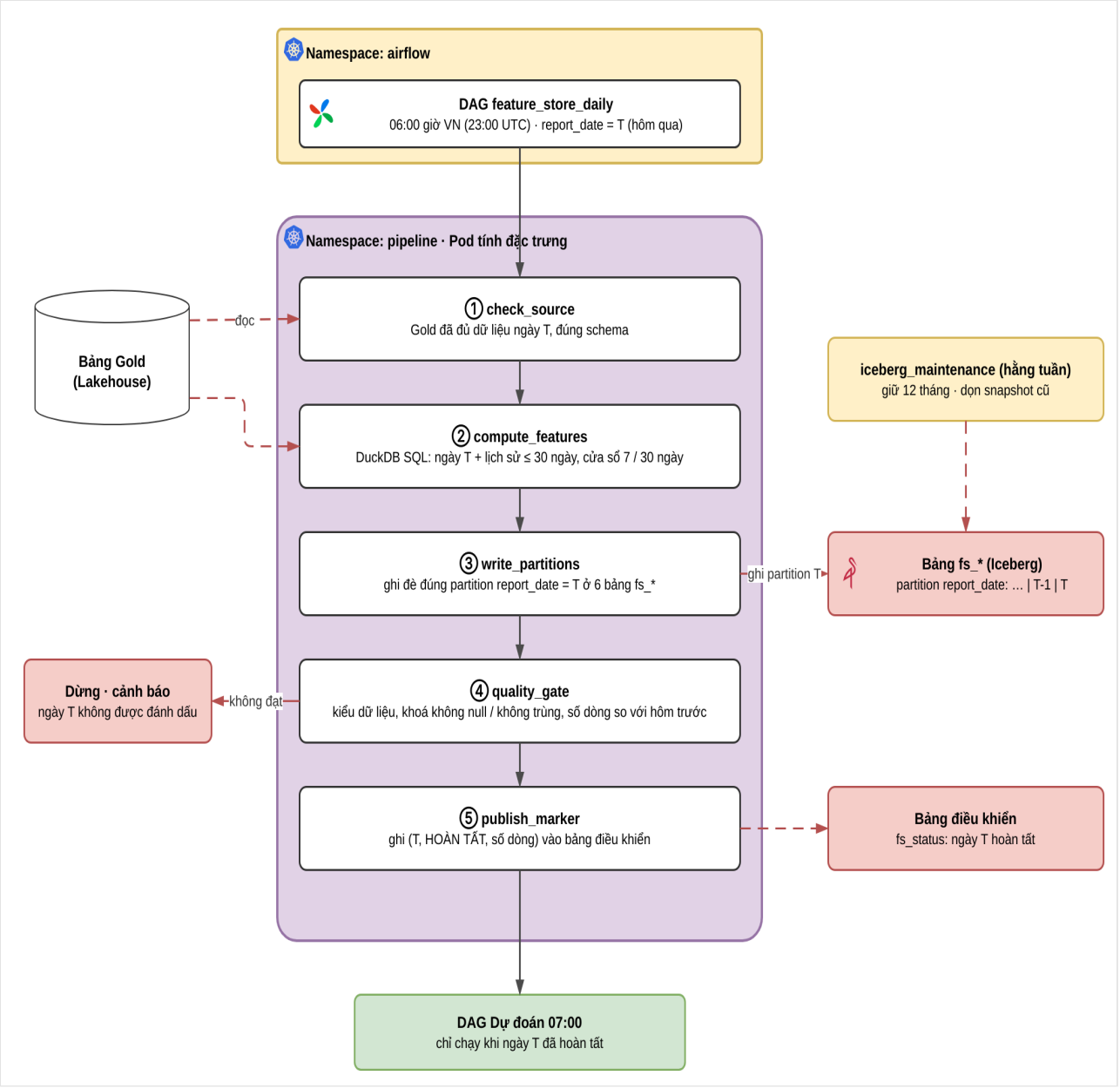
Sau khi mở cổng: Airflow tại `localhost:8080`, MLflow tại `127.0.0.1:5050`, MinIO console tại `localhost:9001`, ArgoCD tại `localhost:8090`.

3. Luồng 1: Gold → Feature Store

3.1 Mục tiêu và nguyên tắc

- Mỗi ngày chỉ tính và ghi **một partition** `report_date = T` cho từng bảng fs_*, không xoá bảng, không tính lại lịch sử.
- Chạy lại ngày T (do lỗi hoặc dữ liệu nguồn sửa) chỉ **thay đúng partition T**, các ngày khác giữ nguyên (idempotent).
- Một ngày chỉ được coi là sẵn sàng khi **vượt cổng chất lượng** và có dấu "hoàn tất"; DAG dự đoán chỉ chạy khi có dấu này.
- T là ngày dữ liệu **hôm qua**; DAG chạy 06:00 giờ Việt Nam, trước DAG dự đoán 07:00.

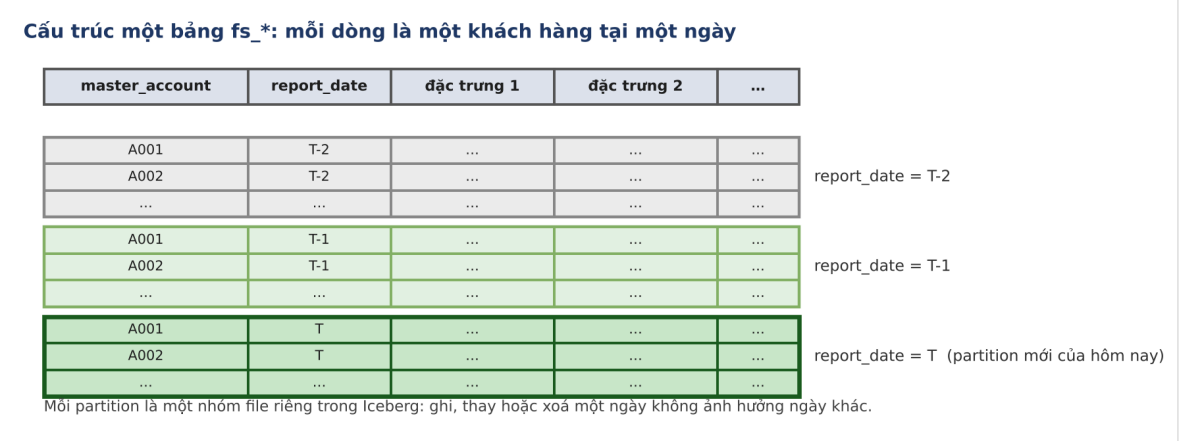
3.2 Sơ đồ luồng



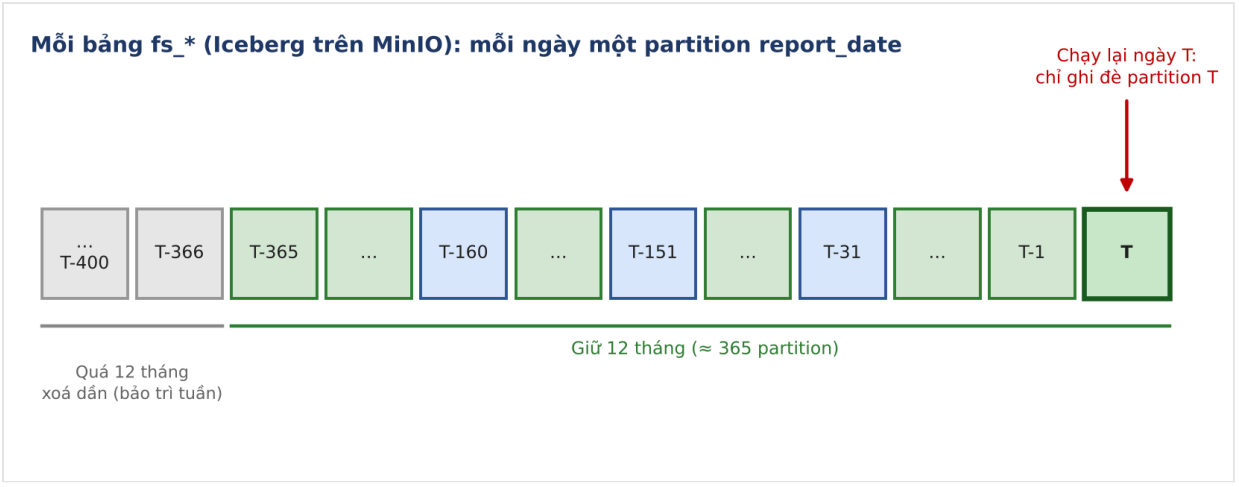
Hình 3.1: Luồng Gold → Feature Store hằng ngày

3.3 Cách lưu: partition theo report_date

Mỗi bảng fs_* là một bảng Iceberg trên MinIO (`s3://warehouse`) để pod Airflow đọc được. Mỗi dòng là một khách hàng tại một ngày. Iceberg chia dữ liệu thành các nhóm file theo `report_date`, nên ghi hoặc thay một ngày không đụng tới ngày khác.



*Hình 3.2: Cấu trúc một bảng fs_**



Hình 3.3: Mỗi ngày một partition, giữ 12 tháng

Bảng 3.1: 6 bảng đặc trưng

Bảng fs_*	Nội dung	Khoá
fs_demographic	Thông tin khách hàng: tuổi tài khoản, segment, …	(master_account, report_date)
fs_trading_behavior	Hành vi giao dịch cửa sổ 7 và 30 ngày	(master_account, report_date)
fs_margin_balance	Dư nợ margin, thay đổi nợ 7 ngày	(master_account, report_date)
fs_asset_balance	Tài sản, tiền	(master_account, report_date)
fs_portfolio	Danh mục nắm giữ (số mã, NAV, …)	(master_account, report_date)
fs_campaign	Tương tác chiến dịch	(master_account, report_date)

> **Nhãn không nằm trong fs_***. Nhãn `loan_label_next` cần nhìn 30 ngày về sau nên được tính riêng từ dòng tiền Gold (bảng `loan_overdue_labels`) và ghép với đặc trưng khi huấn luyện, đánh giá.

3.4 Từng bước hằng ngày

DAG `feature_store_daily`, lịch cron `0 23 * * *` (23:00 UTC = 06:00 giờ VN), tham số `report_date` mặc định là ngày hôm qua. Các task chạy tuần tự.

1 check_source: kiểm tra nguồn Gold đã đủ ngày T

Task đọc ngày dữ liệu lớn nhất ở các bảng nguồn lõi (giao dịch, margin, giá, danh mục, tài khoản). Nếu chưa đến ngày T hoặc sai schema, task lỗi và Airflow thử lại; không tính gì khi nguồn thiếu, tránh ghi partition rỗng hoặc sai.

```
uv run python -m features.check_source --date 2026-01-30
```

2 compute_features: tính đặc trưng ngày T

DuckDB đọc Gold ngày T và lịch sử lùi tối đa 30 ngày (cửa sổ theo ngày lịch 7 và 30, ví dụ debt_change_7d), tính 6 nhóm đặc trưng. Kết quả ghi bảng tạm. Chỉ cần 30 ngày lịch sử nên mỗi lần chạy nhẹ, không phụ thuộc độ dài toàn bộ dữ liệu.

```
uv run python -m features.compute --date 2026-01-30 --out tmp_fs_20260130
```

3 write_partitions: ghi đè partition T ở 6 bảng fs_*

Với từng bảng, ghi vào Iceberg bằng thao tác overwrite có điều kiện `report_date = T`: chưa có partition thì tạo, đã có thì thay. Bảng tạm bị xoá sau khi ghi xong. Vì Iceberg giữ snapshot cũ, chạy lại để lại file cũ chưa dùng; việc dọn nằm ở task bảo trì hằng tuần.

```
uv run python -m features.write --date 2026-01-30 --mode overwrite-partition
```

4 quality_gate: kiểm tra hợp đồng schema và chất lượng trên partition T

Chỉ đọc partition T, mọi kiểm tra là ngưỡng cứng (0 dòng vi phạm): đúng kiểu theo `docs/feature_contract.md`; khoá (master_account, report_date) không null, không trùng; 24 cột đặc trưng không null; giá trị hợp lệ (ví dụ `margin_segment` ∈ {nu, lu, au}, `trade_count_7d ≤ trade_count_30d`). Không đạt thì task đỏ trên Airflow, ngày T không được đánh dấu hoàn tất nên DAG dự đoán không chạy.

```
uv run python -m features.quality --date 2026-01-30
```

5 publish_marker: đánh dấu ngày T hoàn tất

Ghi một dòng vào bảng điều khiển `control.fs_status`: (T, HOÀN TẤT, số dòng từng bảng, thời điểm). DAG dự đoán dùng một cảm biến chờ dòng này. Chạy lại ngày T chỉ cập nhật dòng của T.

```
uv run python -m features.publish --date 2026-01-30
```

6 iceberg_maintenance (hằng tuần): giữ 12 tháng và dọn file

Task riêng chạy mỗi tuần: xoá partition quá 12 tháng, hết hạn snapshot cũ và xoá file mồ côi để dung lượng không phình lên do các lần chạy lại.

```
uv run python -m features.maintenance --retention-months 12 --expire-snapshots --remove-orphan
s
```

3.5 Cách kiểm tra

Lệnh dưới đây in số dòng, số ngày và khoảng `report_date` của 6 bảng fs_* trong run đặc trưng hiện tại. Kỳ vọng là mỗi ngày đúng một nhóm dòng và không có ngày trùng.

```
uv run python docs/screenshots/helpers/show_fs.py
```

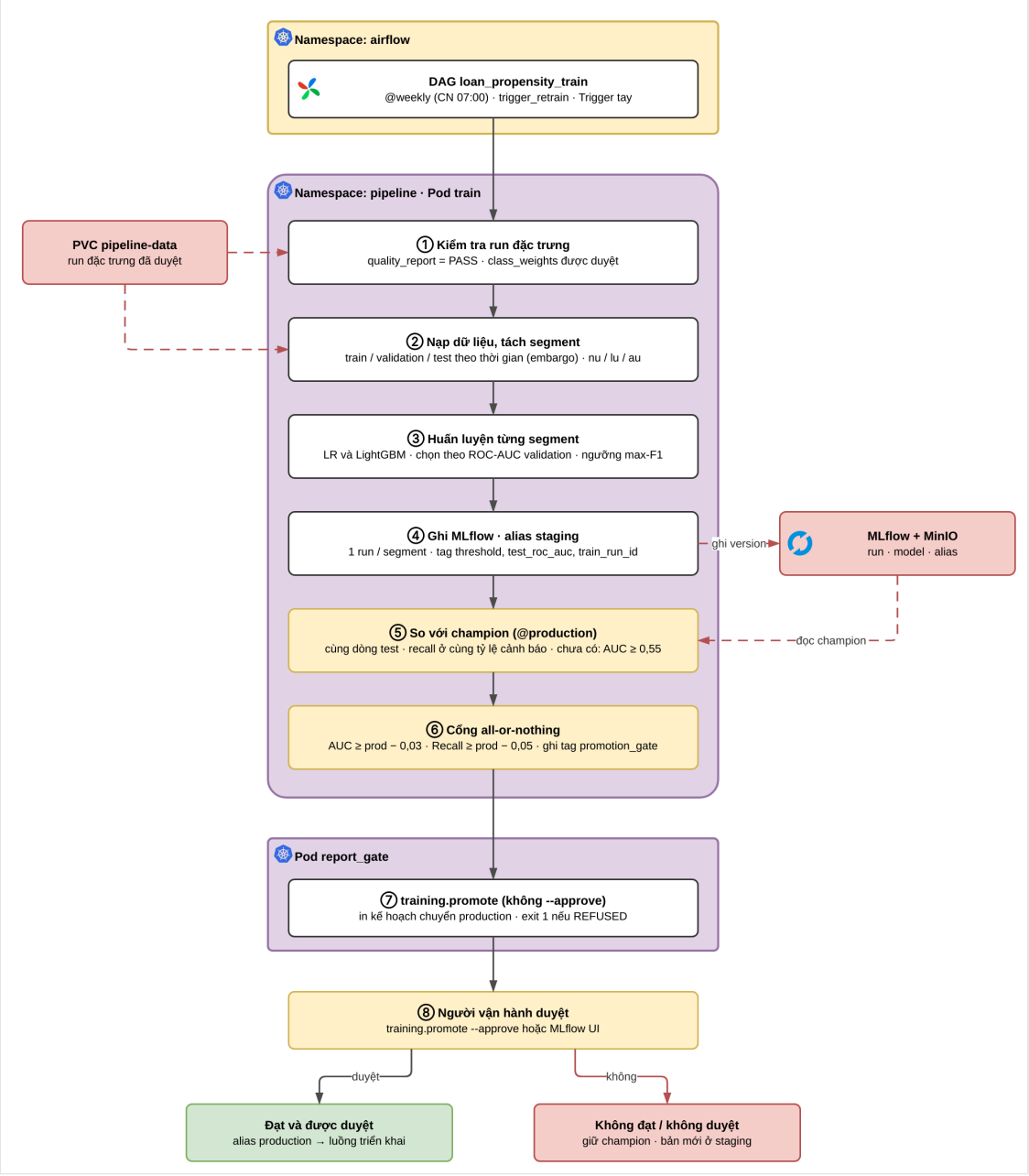
*Bảng 3.2: Kiểm tra nhanh fs_**

Dấu hiệu	Cách đọc
Mỗi bảng có số ngày tăng 1 sau mỗi lần chạy	Đúng: chỉ thêm partition mới
Chạy lại ngày T, số ngày không đổi, số dòng ngày T đổi theo dữ liệu mới	Đúng: partition T được thay
Số ngày giảm hoặc khoảng ngày bị thu hẹp	Sai: bảng bị dựng lại hoặc xoá nhầm

4. Luồng 2: Huấn luyện và đưa model lên production

4.1 Tổng quan

DAG `loan_propensity_train` chạy mỗi Chủ Nhật 07:00 giờ VN, hoặc được DAG dự đoán gọi khi drift ở mức critical, hoặc do người vận hành bấm Trigger. DAG có hai task: **train** (huấn luyện, ghi MLflow, đặt alias staging) rồi **report_gate** (in kết quả cổng champion). **Airflow không bao giờ đặt alias production**; bước đó do người vận hành làm sau khi xem kết quả.



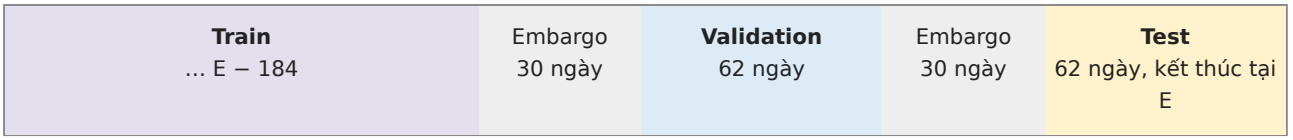
Hình 4.1: Luồng huấn luyện, cổng champion và duyệt production

4.2 Dữ liệu huấn luyện và cách chia thời gian

Mỗi dòng của `training_dataset` là một khách hàng tại một ngày với đặc trưng của ngày đó và nhãn `loan_label_next`. Dữ liệu được chia theo thời gian, không xáo trộn ngẫu nhiên, để mô hình luôn được

đánh giá trên tương lai của dữ liệu nó đã học.

Các mốc không cố định mà tính lùi từ ngày chấm `as_of`. Nhãn của ngày D chỉ biết sau 30 ngày, nên ngày cuối có nhãn là **E = as_of − 30 ngày**. Từ E lùi về: test 62 ngày, embargo 30 ngày, validation 62 ngày, embargo 30 ngày, phần còn lại là train. Mỗi lần huấn luyện lại, cửa sổ trượt theo `as_of` nên model học thêm dữ liệu mới.



Hình 4.2: Chia train / validation / test theo cửa sổ trượt, tính lùi từ ngày có nhãn cuối E

Bảng 4.1: Chia tập dữ liệu

Tập	Theo E	Khi as_of = 2026-01-30	Dùng để
Train	... E − 184	01/02/2025 – 30/06/2025	Học tham số; tính trọng số lớp
Embargo	30 ngày	01/07 – 30/07/2025	Nhãn của train không chạm sang validation
Validation	62 ngày	31/07 – 30/09/2025	Chọn LR hay LightGBM; chọn ngưỡng max-F1
Embargo	30 ngày	01/10 – 30/10/2025	Bỏ qua
Test	62 ngày, kết thúc tại E	31/10 – 31/12/2025	Chấm một lần; đầu vào cổng champion

> Mỗi version ghi tag `data_end` = E (ngày cuối có nhãn mà model đã học và được kiểm tra). Drift dùng ngày này làm tham chiếu, retrain_check dùng nó để biết đã có nhãn mới chưa (mục 5.3, 5.4).

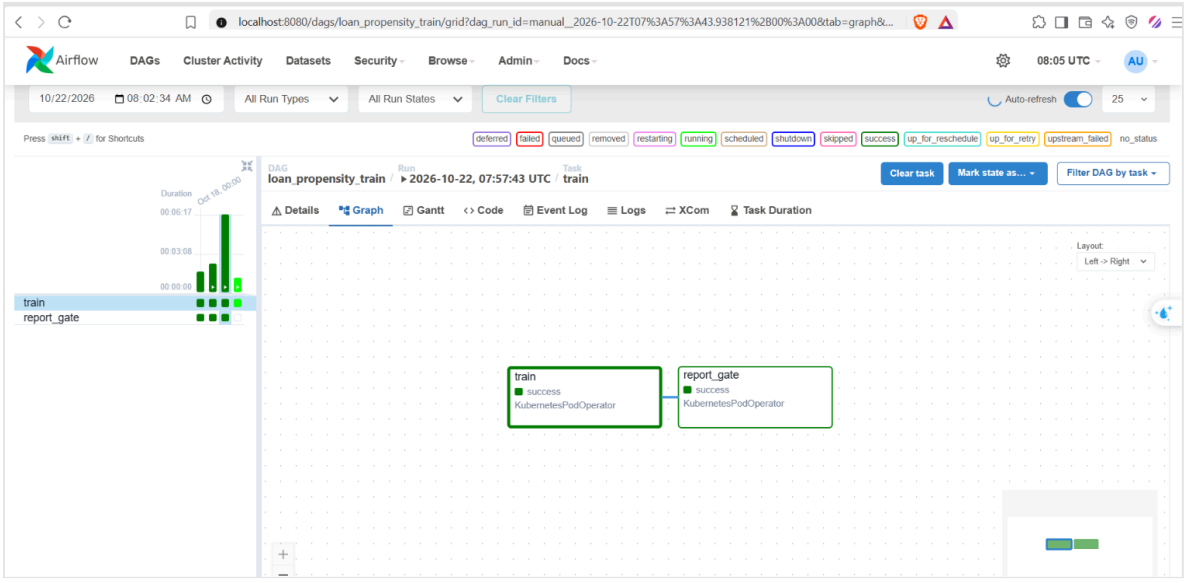
4.3 Từng bước

1 Airflow kích hoạt DAG

Lịch `@weekly` (Chủ Nhật 00:00 UTC). Để chạy ngay, bấm Trigger trên UI hoặc dùng CLI. Task **train** có `retries=1`, giới hạn bộ nhớ 2Gi; task **report_gate** không thử lại (`retries=0`) vì kết quả REFUSED là kết luận, không phải lỗi tạm.

Chạy tay và theo dõi pod:

```
kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_train
kubectl -n pipeline get pods -w
```



Hình 4.3: Airflow: DAG loan_propensity_train gồm hai task train và report_gate, cả hai thành công

2 Pod train kiểm tra run đặc trưng

Pod mount PVC `pipeline-data`, đọc run đặc trưng đã duyệt. Nếu `quality_report.status` không phải PASS thì dừng, không huấn luyện trên dữ liệu chưa đạt chất lượng.

Xem log pod (hoặc tab Logs của task trong Airflow):

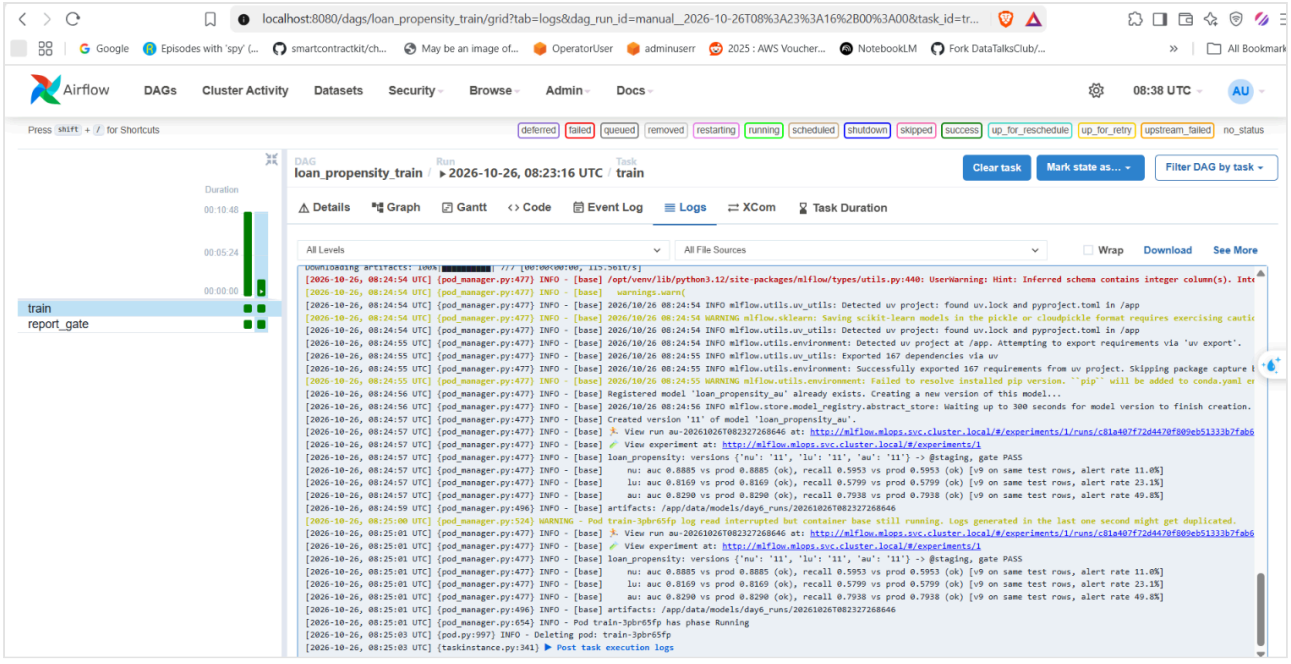
```
kubectl -n pipeline logs -f <tên-pod-train>
```

3 Nạp dữ liệu và tách theo segment

Đọc `training_dataset.parquet`, lọc theo use case (`loan_propensity` gồm nu, lu, au; cấu hình ở `training/usecases.yaml`) và chia thành train / validation / test theo bảng thời gian ở trên. Kiểm tra: không trùng ngày giữa các tập, mỗi tập có đủ hai lớp 0 và 1.

4 Huấn luyện hai loại model cho mỗi segment

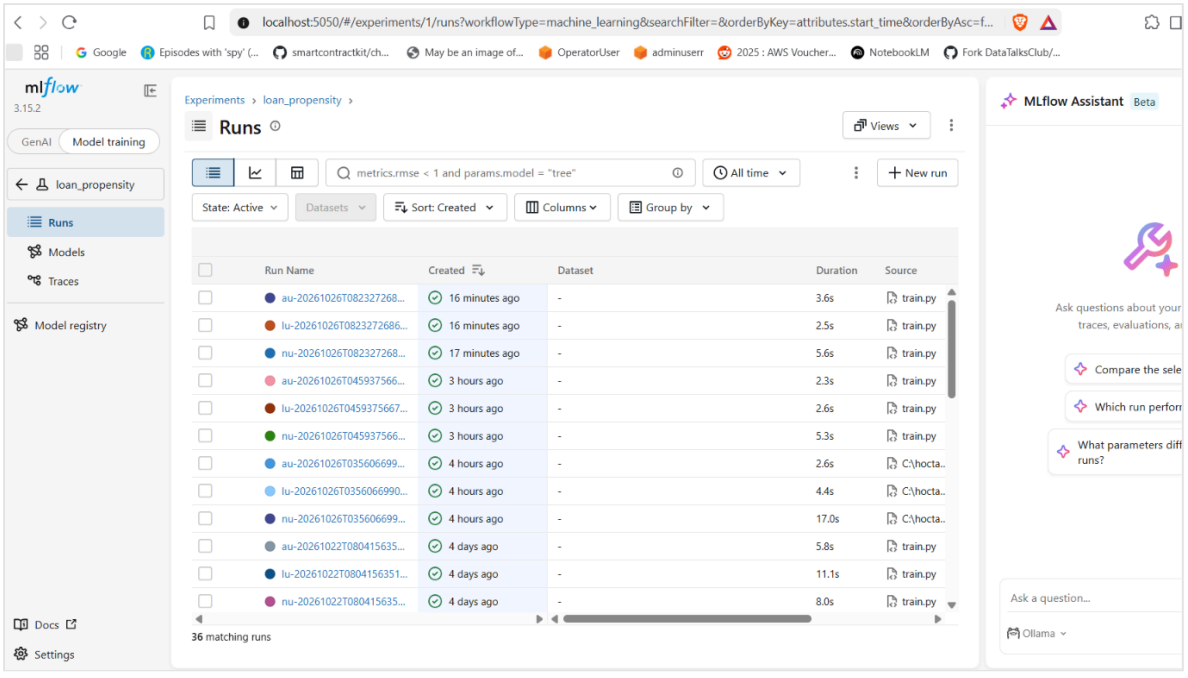
Với từng segment: Logistic Regression và LightGBM, dùng trọng số lớp tính từ tập train (`class_weights.json`) để bù lớp dương hiếm. Chọn model có **ROC-AUC validation** cao hơn. Ngưỡng quyết định là điểm có **F1 lớn nhất** trên validation. Tập test chỉ được chấm một lần, sau khi mọi lựa chọn đã chốt.



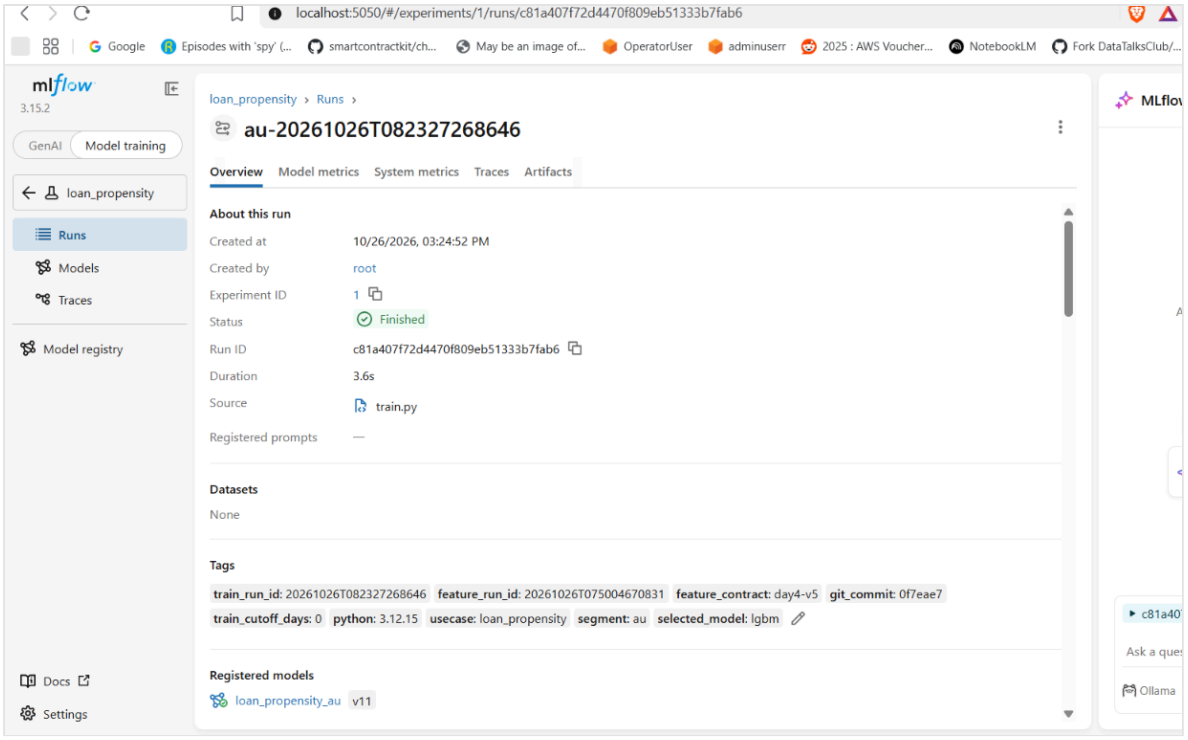
Hình 4.4: Log task train trong Airflow: đăng ký version 11 cho cả ba segment, đặt @staging, cổng PASS (AUC nu 0,8885, lu 0,8169, au 0,8290)

5 Ghi MLflow và đăng ký model

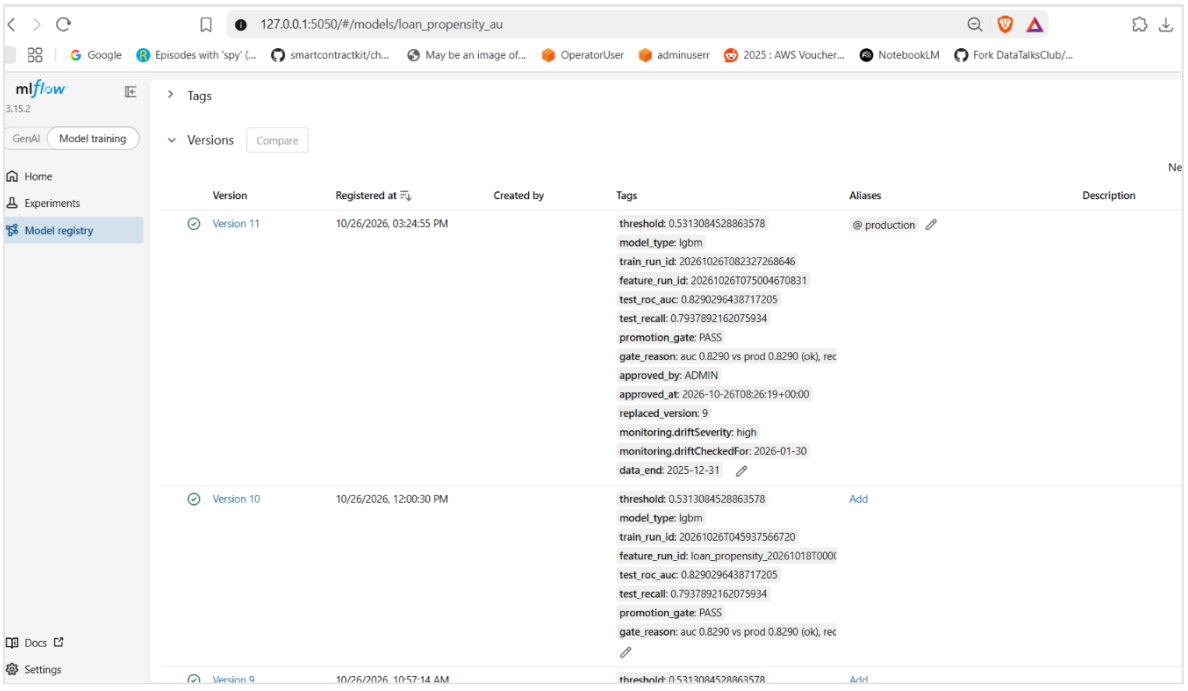
Mỗi segment một run trong experiment `loan_propensity`: tham số, 30 metric, file model (cloudpickle). Model được đăng ký với tên `<usecase>_<segment>` (ví dụ `loan_propensity_au`), version mới nhận alias **@staging** kèm các tag: `threshold`, `model_type`, `train_run_id`, `feature_run_id`, `test_roc_auc`, `test_recall`, `data_end`. File model nằm ở MinIO bucket `mlflow`, metadata ở PostgreSQL.



Hình 4.5: MLflow: danh sách run của experiment loan_propensity (mỗi lần huấn luyện tạo 3 run au / lu / nu)



Hình 4.6: MLflow: chi tiết một run (au), 30 metric và các tag train_run_id, feature_run_id



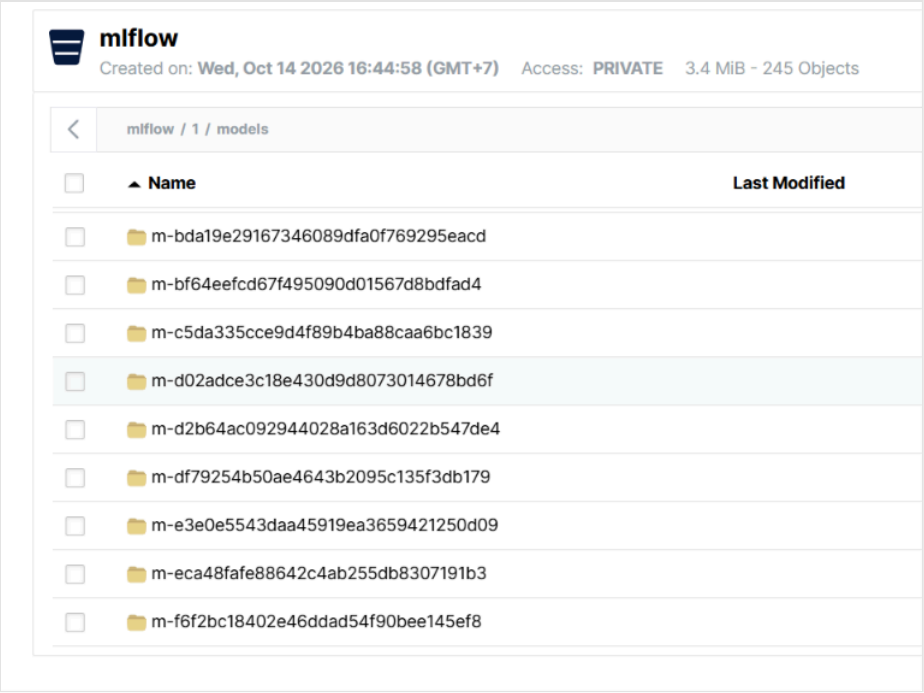
Hình 4.7: MLflow Model registry: loan_propensity_au version 11 ở @production với tag huấn luyện, tag duyệt (approved_by, replaced_version), data_end và tag giám sát drift

6 Kiểm tra file model trên MinIO

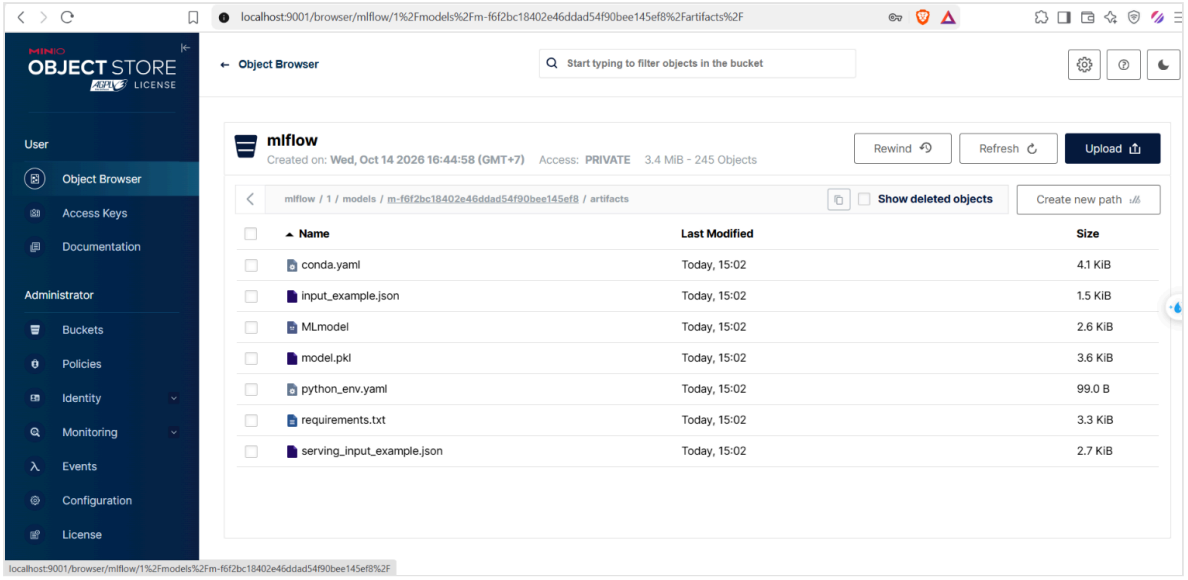
Mỗi version có một thư mục `m-…` trong `mlflow/1/models/`; trong thư mục `artifacts` có `MLmodel`, `model.pkl`, `conda.yaml`, `requirements.txt`… File `MLmodel` cũng là thứ deployer kiểm tra trước khi cho phép triển khai.

Mở http://localhost:9001, bucket mlflow:

```
kubectl -n mlops port-forward svc/minio-console 9001:9001
```



Hình 4.8: MinIO: các thư mục model trong bucket mlflow



Hình 4.9: MinIO: artifact của một version model (MLmodel, model.pkl, requirements.txt...)

7 Cổng champion / challenger

Với mỗi segment, challenger (@staging) và champion (@production) được chấm trên **cùng các dòng test**. Điều kiện đạt: **AUC ≥ max(AUC production − 0,03 ; 0,75)** và **Recall ≥ Recall production − 0,05** (recall so ở cùng tỷ lệ cảnh báo của champion). Chưa có @production thì chỉ cần **AUC ≥ 0,75** (BASELINE_PASS). Sàn 0,75 ngăn model tụt dần qua nhiều lần huấn luyện lại. Cổng là **all-or-nothing**: mọi segment của bài toán phải đạt thì mới đạt. Kết quả ghi vào tag `promotion_gate` (PASS / BASELINE_PASS / FAIL) và `gate_reason` của từng version.

8 Task report_gate in kế hoạch chuyển production

Task chạy `training.promote` **không có** `--approve` nên chỉ in ra bảng: cả ba segment đã cùng một `train_run_id` chưa, cổng PASS hay FAIL, version nào sẽ thay version nào. Nếu bị từ chối thì thoát mã 1 và task đỏ; đó là kết luận “chưa nên chuyển”.

Xem kế hoạch (không thay đổi gì):

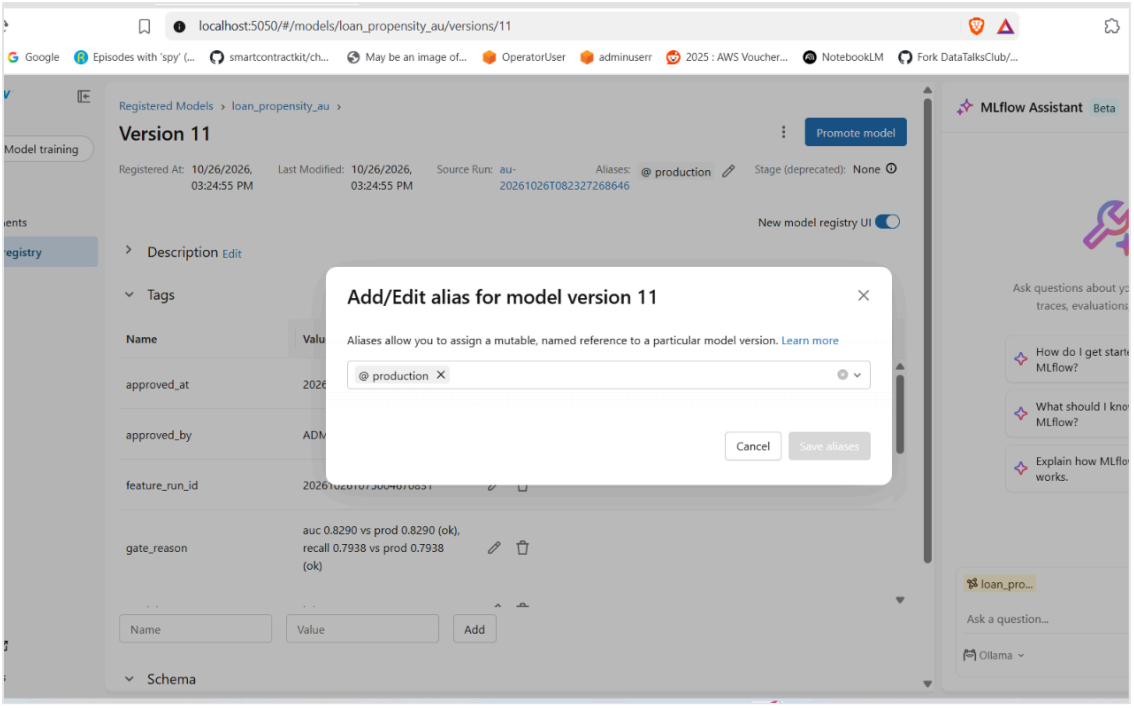
```
uv run python -m training.promote --usecase loan_propensity
```

9 Người vận hành duyệt và đặt alias production

Khi cổng đạt, người vận hành xem metric rồi duyệt bằng một trong hai cách. Cách 1 (khuyến nghị): lệnh `--approve`; lệnh từ chối nếu ba segment @staging không cùng một lần huấn luyện hoặc cổng chưa đạt, và ghi tag `approved_by`, `approved_at`, `replaced_version`. Cách 2: tự đặt alias trên MLflow UI (không có các kiểm tra trên, nên chỉ dùng khi hiểu rõ).

Cách 1: dòng lệnh

```
uv run python -m training.promote --usecase loan_propensity --approve
```



Hình 4.10: MLflow UI: hộp thoại Add/Edit alias của version 11, nơi đặt alias production (bản duyệt bằng tay thay cho lệnh --approve)

> **Quay lại version cũ (rollback)**: đặt lại alias `production` về version cũ (UI hoặc API MLflow). Deployer sẽ thấy alias đổi và commit lại manifest (luồng 4). Version cũ không bị xoá nên luôn có thể quay lại.

4.4 Áp dụng cho overdue_risk và campaign_response

Hai bài toán còn lại đi qua đúng các bước trên với DAG `overdue_risk_train` và `campaign_response_train` (tham số `--usecase` khác nhau). Điểm khác nằm ở dữ liệu đầu vào và số model:

Bảng 4.2: Khác biệt khi huấn luyện hai bài toán còn lại

	overdue_risk	campaign_response
Dữ liệu	`training_dataset`, chỉ dòng `overdue_eligible = 1` (có dư nợ hoặc lịch sử margin, chưa quá hạn tại D)	`campaign_training_dataset`, mỗi dòng là một lần gửi chiến dịch
Nhãn	`overdue_label_next_30d`	`campaign_response_label_14d`

Đặc trưng	24 đặc trưng tài khoản như loan_propensity	16 đặc trưng số + kênh, loại ưu đãi, segment (dạng phân loại), lãi suất ưu đãi
Model	2 model: lu, au	1 model chung, segment là đặc trưng
Cổng	All-or-nothing trên 2 segment, cùng ngưỡng 0,03 / 0,05, sàn AUC 0,75	Một model nên cổng chỉ xét model đó

4.5 Kết quả và dữ liệu sinh ra

Bảng 4.3: Sản phẩm của luồng huấn luyện

Sản phẩm	Nơi lưu	Ai dùng tiếp
Version mới của các model của bài toán (3 model với loan_propensity), alias staging	MLflow + MinIO	Cổng champion, người vận hành duyệt
Tag threshold, test_roc_auc, test_recall	Tag của version	Dự đoán (ngưỡng), đánh giá (so với AUC lúc huấn luyện)
Tag promotion_gate, gate_reason	Tag của version	Task report_gate, người vận hành
Alias production (sau khi duyệt)	MLflow	Dự đoán batch và deployer

4.6 Thế nào là model tốt hơn

"Tốt hơn" được định nghĩa khác nhau ở ba thời điểm, mỗi lần trả lời một câu hỏi riêng và dùng tập dữ liệu riêng:

Bảng 4.4: Ba lần so sánh model

Thời điểm	So sánh	Dữ liệu	Tiêu chí	Kết quả
1. Khi huấn luyện	Logistic Regression và LightGBM của cùng một segment	Validation	ROC-AUC validation cao hơn	Model thắng được đăng ký, ngưỡng chọn trên validation
2. Cổng champion / challenger	Version mới (@staging) và version đang chạy (@production)	Cùng các dòng test	AUC ≥ max(AUC prod − 0,03 ; 0,75) **và** Recall ≥ Recall prod − 0,05; phải đạt ở **mọi** segment	PASS / BASELINE_PASS / FAIL, rồi người vận hành duyệt
3. Khi đã chạy	Version production tại thời điểm huấn luyện và trên dữ liệu mới	Dữ liệu mới có nhãn	AUC thực tế so với `test_roc_auc`: tụt > 0,05 là WARN, > 0,10 là RETRAIN	OK / WARN / RETRAIN trong bảng eval

Bảng 4.5: Các chỉ số dùng để so sánh

Chỉ số	Ý nghĩa	Dùng ở đâu
ROC-AUC	Xác suất model xếp một khách hàng dương cao hơn một khách hàng âm; 0,5 là ngẫu nhiên, 1 là hoàn hảo. Không phụ thuộc ngưỡng	Chọn model, cổng, đánh giá hằng tuần
Recall tại cùng tỷ lệ cảnh báo	Trong số khách hàng thật sự dương, bao nhiêu phần trăm nằm trong top X% được model cảnh báo. Hai model được so với cùng X nên ngưỡng khác nhau không làm lệch kết quả	Cổng champion / challenger

Precision	Trong số khách hàng bị cảnh báo, bao nhiêu phần trăm thật sự dương (chi phí liên hệ nhầm)	Báo cáo, chọn ngưỡng
F1	Trung bình điều hoà của precision và recall; ngưỡng quyết định là điểm có F1 lớn nhất trên validation	Chọn ngưỡng (`threshold`)
PR-AUC	Diện tích dưới đường precision – recall, nhạy với lớp dương hiếm	Ghi trong MLflow để tham khảo

Ví dụ lần 1 (chọn model trong lần huấn luyện loan_propensity):

Bảng 4.6: Chọn giữa Logistic Regression và LightGBM theo AUC validation

Segment	LR: AUC val / test	LightGBM: AUC val / test	Model được chọn
nu	0,81 / 0,87	0,83 / 0,89	LightGBM
lu	0,81 / 0,82	0,80 / 0,80	Logistic Regression
au	0,73 / 0,75	0,80 / 0,83	LightGBM

Ví dụ lần 2 (cổng). Champion au có AUC 0,83, recall 0,79 → version mới cần AUC ≥ max(0,80 ; 0,75) = 0,80 và recall ≥ 0,74. Nếu champion chỉ còn 0,77 thì ngưỡng là max(0,74 ; 0,75) = 0,75, không thấp hơn được nữa. Cổng chỉ làm version đủ điều kiện; đặt production vẫn cần người vận hành duyệt.

4.7 Chất lượng model

Chất lượng được đo bằng ROC-AUC trên tập validation (dùng để chọn model và ngưỡng) và tập test (chấm đúng một lần), theo cách chia thời gian có embargo ở mục 4.2. Mỗi segment huấn luyện Logistic Regression và LightGBM, chọn model có ROC-AUC validation cao hơn (mục 4.3).

Bảng 4.7: ROC-AUC của sáu model

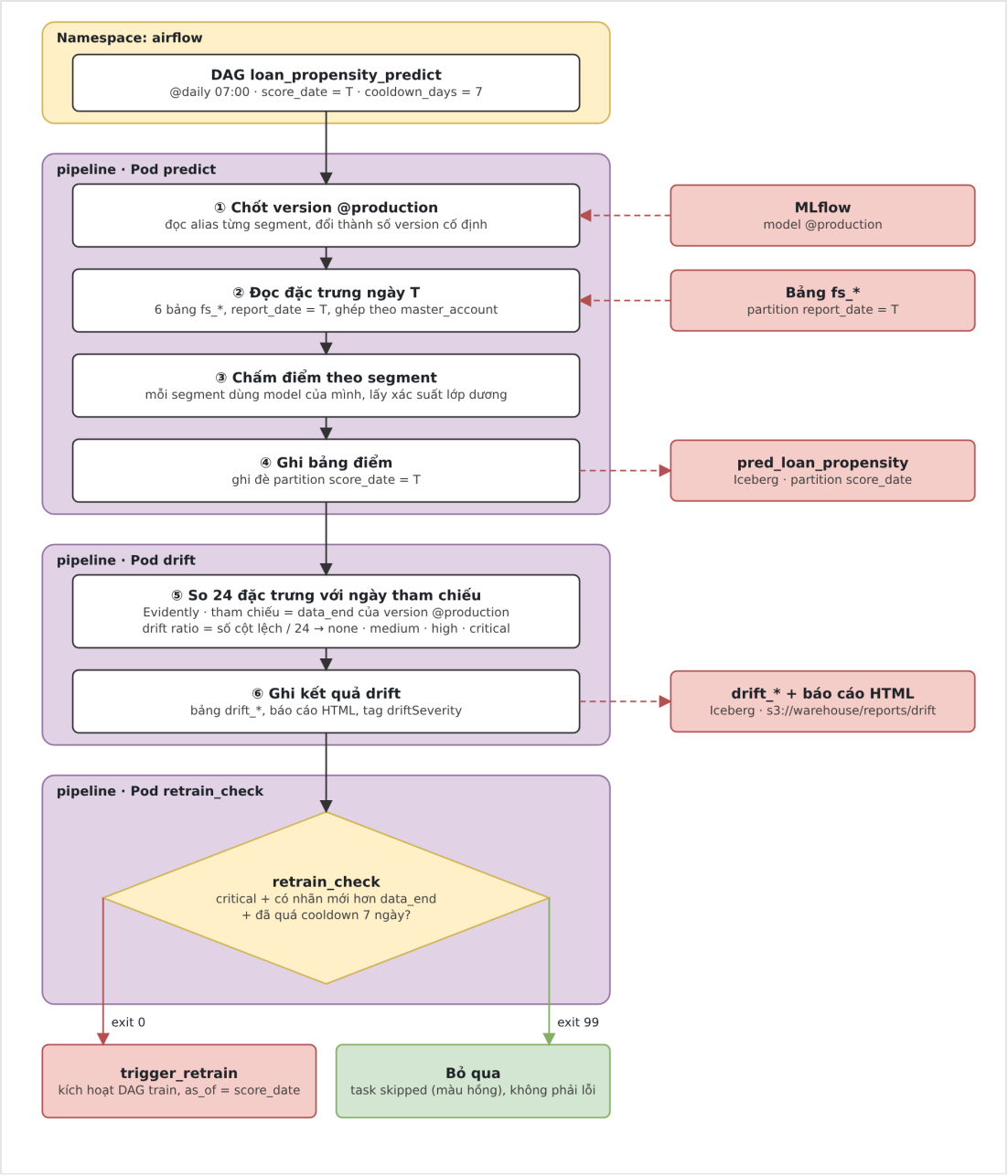
Bài toán	Segment	Model được chọn	ROC-AUC validation	ROC-AUC test
loan_propensity	nu	LightGBM	0,83	**0,89**
loan_propensity	lu	Logistic Regression	0,81	**0,82**
loan_propensity	au	LightGBM	0,80	**0,83**
overdue_risk	lu	Logistic Regression	0,92	**0,95**
overdue_risk	au	LightGBM	0,84	**0,87**
campaign_response	chung	LightGBM	0,89	**0,91**

> **Vì sao AUC test cao hơn validation?** Hai tập thuộc hai giai đoạn khác nhau nên tập khách hàng và tỷ lệ nhãn dương khác nhau; test chỉ chấm một lần sau khi đã chốt model nên không rò rỉ.
> **overdue_risk lu đạt 0,95** vì các đặc trưng sát hạn mức (`margin_utilization_ratio`, `over_limit_streak`) là tín hiệu sớm của quá hạn; nhãn chỉ tính từ D+1 và chỉ chấm khách hàng chưa quá hạn tại D nên không rò rỉ nhãn.

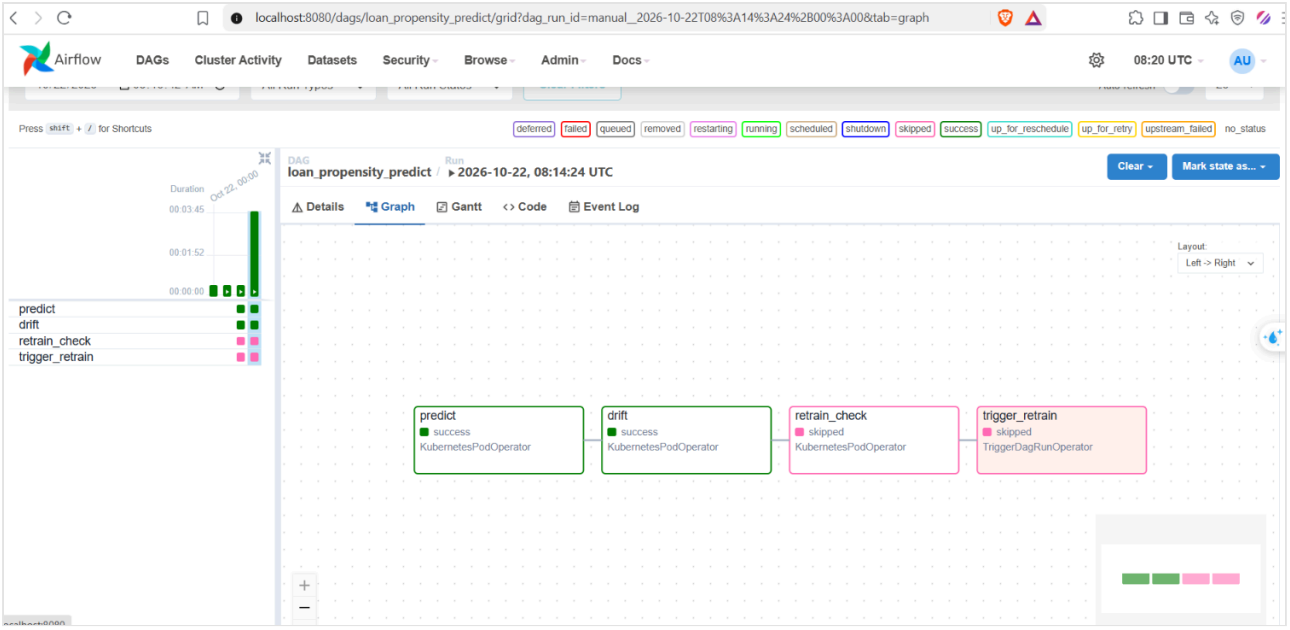
5. Luồng 3: Dự đoán batch và giám sát drift

5.1 Tổng quan

DAG `loan_propensity_predict` chạy `@daily` (00:00 UTC = 07:00 giờ VN) với tham số `score_date` (mặc định 2026-01-30) và `cooldown_days` (mặc định 7). Bốn task nối tiếp: **predict → drift → retrain_check → trigger_retrain**. Mỗi task là một pod mới từ image `mlops-pipeline`.



Hình 5.1: Luồng dự đoán batch, drift và quyết định huấn luyện lại



Hình 5.2: Airflow: đồ thị DAG loan_propensity_predict. predict và drift thành công; retrain_check và trigger_retrain bị bỏ qua (màu hồng)

5.2 Dự đoán: predict

1 Chốt version @production của từng segment

Task đọc alias `production` của `loan_propensity_au / lu / nu` **một lần**, đổi thành số version cố định rồi nạp đúng version đó từ MLflow. Nhờ vậy nếu người vận hành đổi alias giữa chừng, một lần chạy không trộn hai version. Segment chưa có @production thì không được chấm. Dự đoán batch nạp model trực tiếp từ MLflow, **không đi qua KServe**.

Chạy tay một ngày cụ thể:

```
kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_predict --conf
'{"score_date":"2026-01-30"}'
```

2 Đọc đặc trưng ngày chấm

Đọc 6 bảng fs_* với `report_date = score_date` và ghép theo (master_account, report_date) thành một dòng đủ đặc trưng cho mỗi tài khoản. Đây chính là partition ngày T của các bảng fs_*.

3 Chấm điểm theo segment

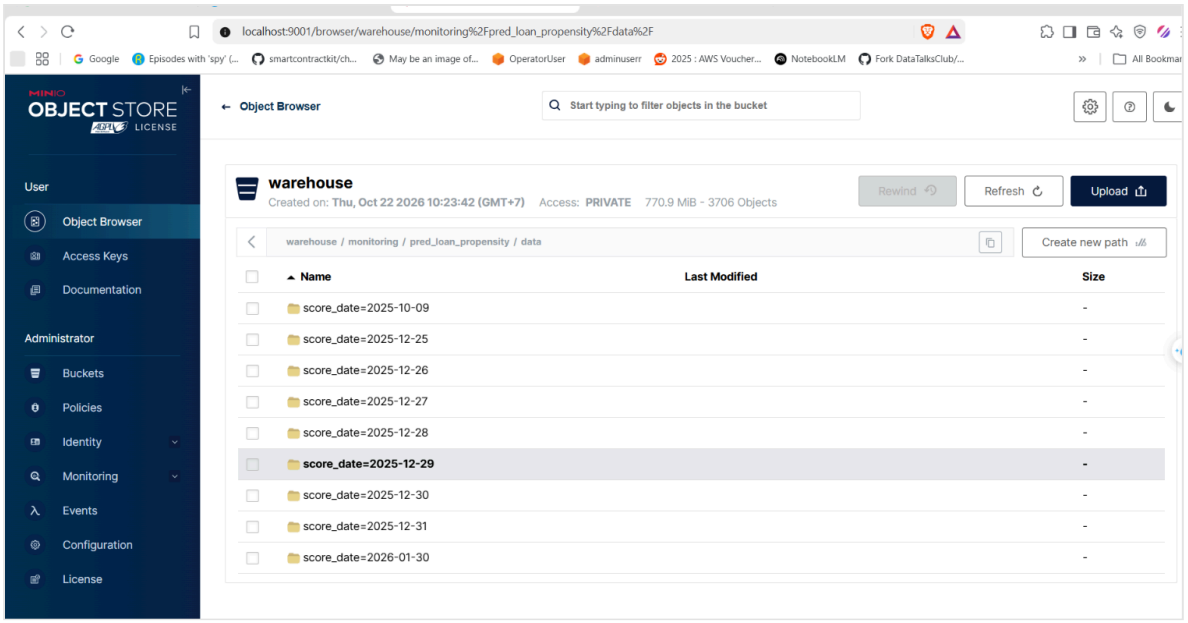
Mỗi khách hàng được chấm bằng model của segment mình, lấy xác suất lớp dương làm `score`. Ngưỡng (tag `threshold`) không áp dụng ở bước này: bảng lưu điểm thô để dùng cho nhiều mục đích.

4 Ghi bảng điểm Iceberg

Ghi vào `monitoring.pred_loan_propensity` (catalog PyIceberg trong PostgreSQL DB `iceberg_catalog`, dữ liệu ở `s3://warehouse`), partition identity theo `score_date`. Ghi bằng `overwrite` có điều kiện `score_date = ngày chấm` nên chạy lại cùng ngày thay đúng partition đó, các ngày khác không đổi. Cột: master_account, score_date, segment, score, model_name, model_version, feature_run_id, scored_at. Ví dụ ngày 2026-01-30: 4731 dòng (au 1807, lu 1965, nu 959), tất cả do version 11 chấm.

Kiểm tra số dòng mỗi partition (cần port-forward đang mở):

```
kubectl -n pipeline get pods
uv run python docs/screenshots/helpers/show_monitoring.py
```



Hình 5.3: MinIO: partition score_date=... của bảng pred_loan_propensity, mỗi ngày một file parquet

5.3 Giám sát drift: drift

Data drift là hiện tượng phân phối đặc trưng đầu vào của ngày hiện tại khác với dữ liệu mà model đã học. Khi đó model vẫn trả điểm bình thường (không có lỗi) nhưng điểm kém tin cậy hơn. Drift được theo dõi hằng ngày vì đặc trưng có ngay, trong khi nhãn thật chỉ chín sau 30 ngày, nên đây là tín hiệu cảnh báo sớm.

Bảng 5.1: Cách giám sát data drift

Thành phần	Nội dung
Dữ liệu tham chiếu	Đặc trưng ngày `data_end` của version @production (2025-12-31 với version 11); mốc này dời theo mỗi lần duyệt production
Dữ liệu hiện tại	Đặc trưng của ngày `score_date`
Phạm vi	24 đặc trưng đầu vào của model (không gồm khoá và nhãn)
Phép so sánh	Evidently `DataDriftPreset` so từng cột; phép thử được chọn theo kiểu cột và số mẫu (khoảng cách giữa hai phân phối hoặc p-value). Mỗi cột có method, score, threshold và cờ drifted
Chỉ số tổng	**Drift ratio** = số cột lệch / 24
Mức độ	none (ratio = 0), medium (≤ 0,20), high (≤ 0,50), **critical** (> 0,50)
Hành động	Chỉ **critical** mới yêu cầu huấn luyện lại, khi đã có nhãn mới hơn `data_end` và quá cooldown 7 ngày; các mức khác chỉ ghi nhận. Huấn luyện lại vẫn dừng ở @staging
Nơi lưu	Bảng `monitoring.drift_<usecase>` (từng cột, partition `current_date`), báo cáo HTML ở `s3://warehouse/reports/drift/…`, tag `monitoring.driftSeverity` của version production

Bảng 5.2: Drift khác đánh giá thực tế thế nào

	Drift (luồng 3)	Đánh giá thực tế (luồng 5)
Đối tượng	Đầu vào: phân phối đặc trưng	Đầu ra: model dự đoán đúng đến đâu
Cần nhãn thật	Không	Có (chờ nhãn chín 14–30 ngày)
Tần suất	Hằng ngày	Hằng tuần
Dùng để	Cảnh báo sớm, có thể tự kích hoạt huấn luyện	Xác nhận chất lượng thật; WARN / RETRAIN

5 So phân phối 24 đặc trưng

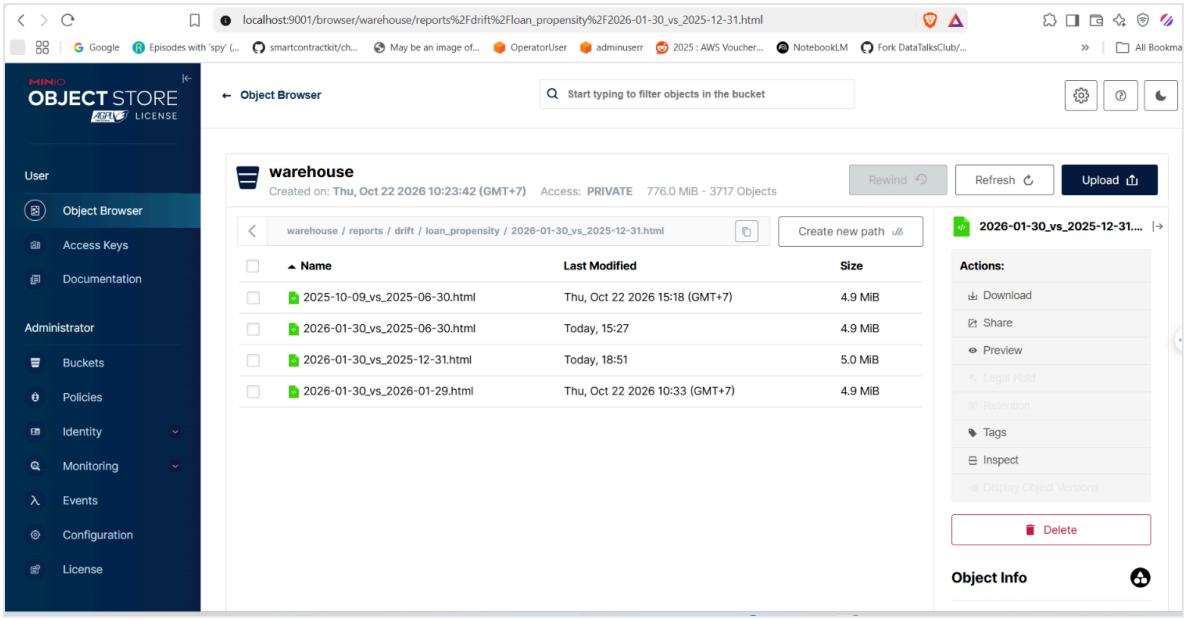
Evidently (`DataDriftPreset`) so phân phối 24 đặc trưng đầu vào của model, ngày hiện tại với ngày tham chiếu `data_end` của version @production. **Drift ratio** = số cột lệch / 24. Mức độ: none khi ratio = 0, medium khi ≤ 0,20, high khi ≤ 0,50, **critical** khi > 0,50. Ví dụ 2026-01-30 so với 2025-12-31: 12/24 (50,0%) = high.

Chạy riêng task drift để thử:

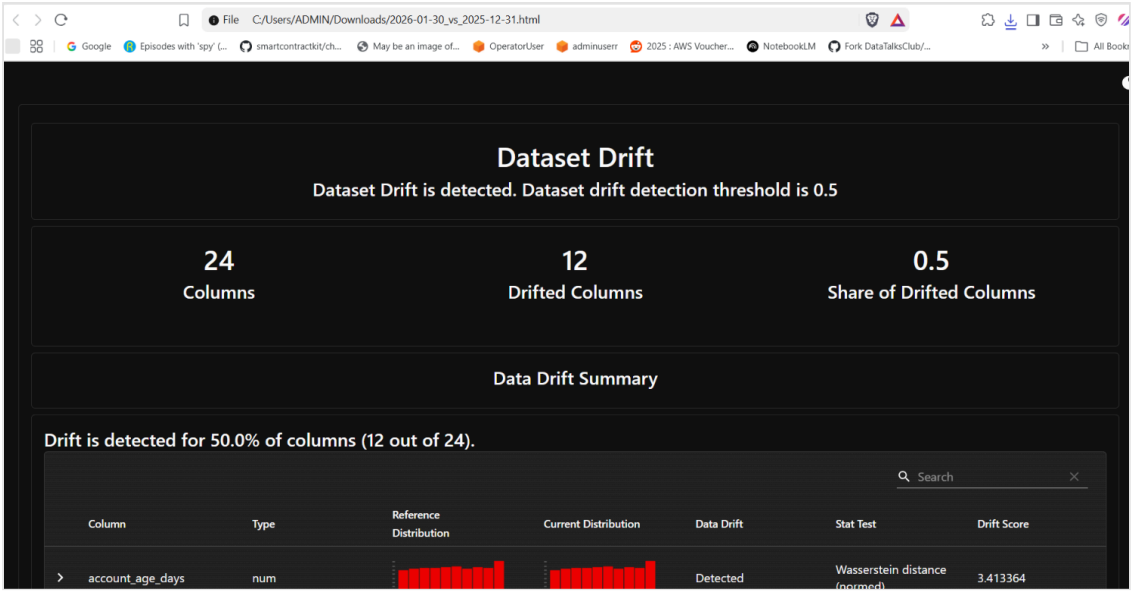
```
kubectl -n airflow exec deploy/airflow -- airflow tasks test loan_propensity_predict drift 2026-01-30
```

6 Ghi kết quả drift

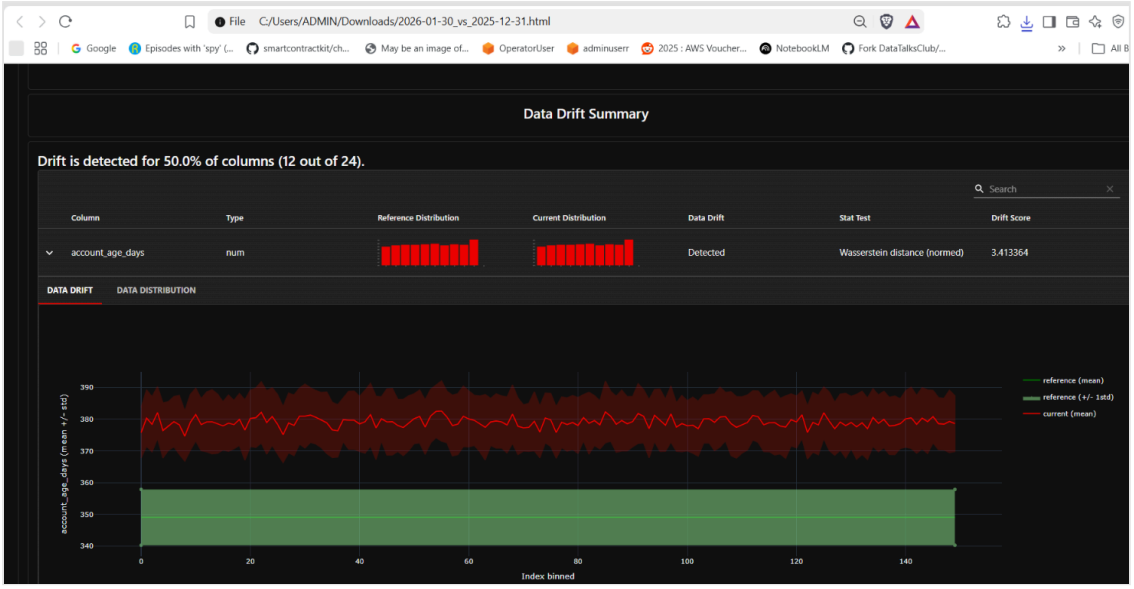
Ghi một dòng cho mỗi cột vào `monitoring.drift_loan_propensity` (partition `current_date`, ghi đè theo ngày), lưu báo cáo HTML ở `s3://warehouse/reports/drift/loan_propensity/<ngày>_vs_<tham chiếu>.html`. Với lần kiểm tra chính thức, mỗi version @production được gắn tag `monitoring.driftSeverity` và `monitoring.driftCheckedFor`.



Hình 5.4: MinIO: các báo cáo HTML drift; file 2026-01-30_vs_2025-12-31.html là lần kiểm tra chính thức



Hình 5.5: Báo cáo Evidently: tổng kết drift (12/24 cột lệch, 50,0%)



Hình 5.6: Báo cáo Evidently: cột account_age_days, phân phối tham chiếu và hiện tại

> **Vì sao tham chiếu theo version production?** Nếu cố định một ngày cũ (ví dụ 2025-06-30, cách 7 tháng) thì các cột tăng tự nhiên như `account_age_days`, `NAV` luôn lệch: 19/24 cột, critical, dù AUC thực tế vẫn tốt, tức cảnh báo giả và huấn luyện lại liên tục. So với `data_end` (cách 30 ngày) chỉ còn 12/24, mức high.

5.4 Quyết định huấn luyện lại: retrain_check

7 Điều kiện huấn luyện lại

Task chạy `pipeline.drift --should-retrain`. Huấn luyện lại khi đủ ba điều kiện: (1) drift **critical**; (2) **có nhãn mới** hơn `data_end` của version @production, nếu không huấn luyện lại chỉ tạo ra đúng model cũ; (3) đã quá **cooldown 7 ngày**. Mã thoát **0** = huấn luyện lại, **99** = bỏ qua (task màu hồng “skipped”, không phải lỗi).

8 trigger_retrain

Chỉ chạy khi retrain_check thoát mã 0: dùng TriggerDagRunOperator kích hoạt `loan_propensity_train` với `as_of = score_date`. Từ đó luồng 2 chạy như bình thường, vẫn cần người vận hành duyệt mới có alias production.

> Trong ảnh đồ thị DAG ở mục 5.1, hai task cuối màu hồng (skipped) vì retrain_check thoát mã 99, không phải lỗi.

5.5 Áp dụng cho overdue_risk và campaign_response

Bảng 5.3: Dự đoán và drift của hai bài toán còn lại

	overdue_risk	campaign_response
DAG	`overdue_risk_predict`, @daily	`campaign_response_predict`, @daily
Dòng được chấm ngày T	Khách hàng đủ điều kiện (`overdue_eligible = 1`) tại T, segment lu / au	Các lần gửi chiến dịch DELIVERED trong ngày T
Bảng điểm	`monitoring.pred_overdue_risk`, partition `score_date`	`monitoring.pred_campaign_response`, partition `score_date`
Drift, retrain_check	Cùng cách tính, ngưỡng và cooldown trong `pipeline/monitoring.yaml`	Cùng cách tính trên đặc trưng của model campaign

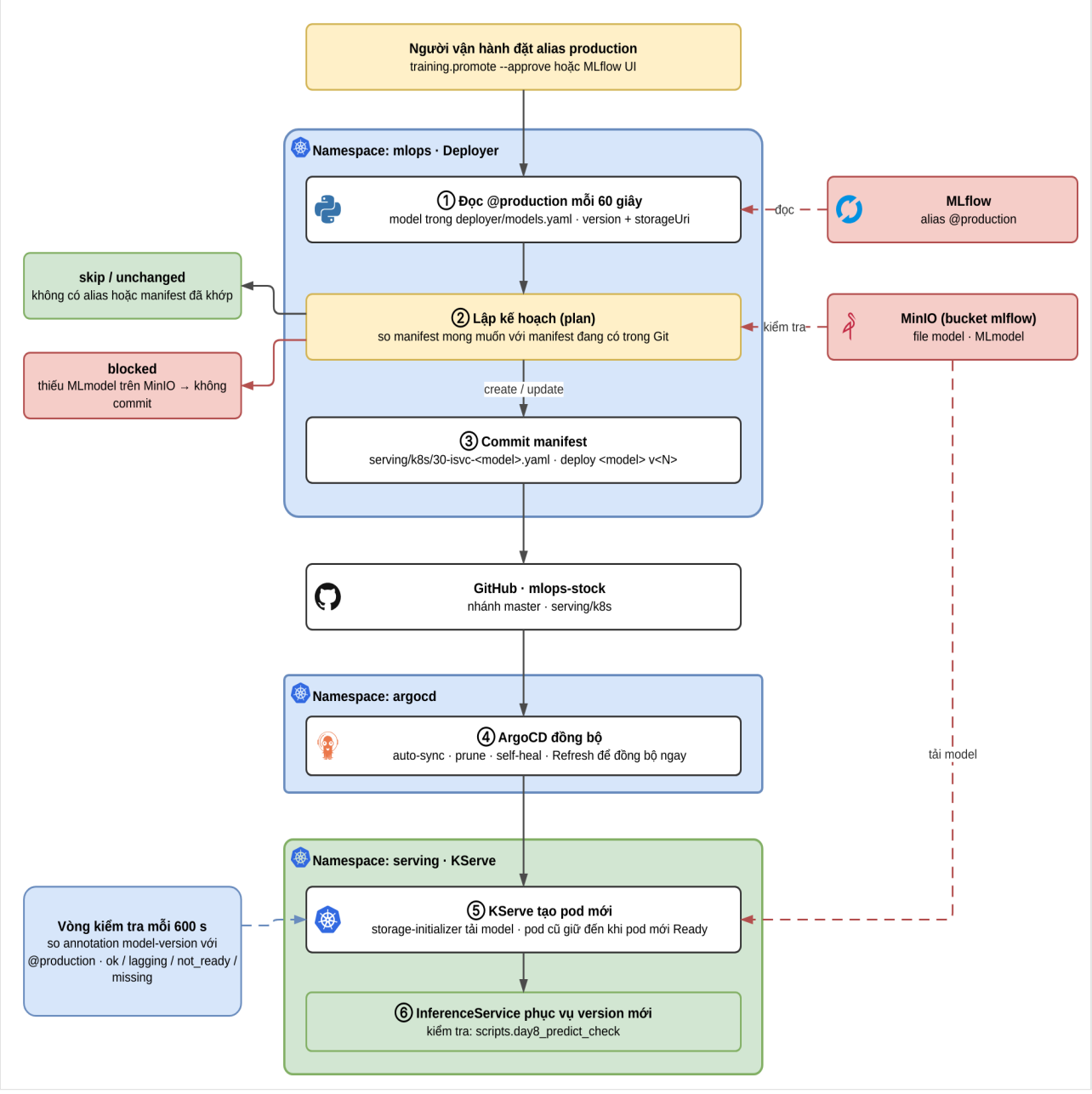
Bảng 5.4: Sản phẩm của luồng dự đoán

Sản phẩm	Nơi lưu	Cách ghi
Điểm dự đoán	Iceberg pred_loan_propensity, partition score_date	Ghi đè partition ngày chấm
Kết quả drift từng cột	Iceberg drift_loan_propensity, partition current_date	Ghi đè theo ngày
Báo cáo HTML	s3://warehouse/reports/drift/loan_propensity/	Mỗi cặp ngày một file
Tag drift của version production	MLflow	Cập nhật mỗi lần kiểm tra chính thức

6. Luồng 4: Triển khai model lên KServe

6.1 Tổng quan

Việc triển khai bắt đầu khi alias `production` thay đổi (do người vận hành). Deployer (Deployment thường trực trong namespace `mlops`) đọc alias, ghi manifest vào Git, ArgoCD kéo manifest vào cụm, KServe tạo pod mới. Đây là GitOps: mọi thay đổi trong cụm đều có commit tương ứng, và sửa tay trong cụm bị ArgoCD hoàn nguyên.



Hình 6.1: Luồng triển khai: alias production → deployer → GitHub → ArgoCD → KServe

6.2 Vai trò của KServe: phục vụ ai, để làm gì

Hệ thống có hai đường dùng model, cùng một artifact và cùng version @production:

Bảng 6.1: Hai đường dùng model

	Dự đoán batch (luồng 3)	KServe (luồng 4)
Phục vụ ai	Bộ phận kinh doanh và chăm sóc khách hàng: danh sách khách hàng cần liên hệ mỗi sáng	Các ứng dụng và dịch vụ cần điểm cho **một khách hàng ngay lúc cần**: màn hình tư vấn viên / CRM, app giao dịch, hệ thống chiến dịch
Mục đích	Chấm toàn bộ khách hàng một lần mỗi ngày, ghi bảng `pred_*`	Dự đoán trực tuyến (online serving): trả điểm qua REST theo yêu cầu, không chờ lô ngày kế tiếp
Điểm vào	Đặc trưng từ bảng fs_* theo `report_date`	Đặc trưng gửi kèm trong request (về sau lấy từ online feature store)
Trạng thái	Đang chạy hằng ngày	Hạ tầng triển khai và kiểm chứng xong; tích hợp gọi từ ứng dụng là hướng phát triển

Giá trị của việc dựng sẵn KServe trong phạm vi MLOps: (1) model **đi cùng một quy trình** (cổng champion, người duyệt, GitOps) rồi mới phục vụ, nên điểm trực tuyến và điểm batch dùng đúng một version; kiểm tra thực tế cho độ lệch bằng 0 (`max_abs_diff` = 0); (2) đổi version hoặc quay lại version cũ chỉ cần đặt lại alias, mọi thay đổi có commit Git; (3) khi có nhu cầu trực tuyến, chỉ cần bổ sung phía gọi mà không phải đổi cách triển khai model.

> **Hướng phát triển (ngoài phạm vi tài liệu này).** Để phục vụ trực tuyến đầy đủ cần thêm: kho đặc trưng trực tuyến (Feast + Redis) để lấy đặc trưng mới nhất theo khách hàng; API gateway xác thực và định tuyến request tới InferenceService; autoscaling và triển khai canary; giám sát độ trễ và tỷ lệ lỗi. Các phần này chưa thực hiện; luồng triển khai hiện tại là nền tảng để gắn chúng vào.

6.3 Từng bước

1 Chuẩn bị deployer

Deployer đọc `GITHUB_TOKEN` (quyền ghi repo) từ secret, truy cập MLflow và MinIO qua service nội bộ của cụm. Danh sách model cần phục vụ khai báo ở `deployer/models.yaml` (repo `Cngdz/mlops-stock`, nhánh `master`, namespace `serving`); mỗi model một InferenceService (khoảng 1 GB RAM) nên chỉ thêm model khi cụm còn đủ tài nguyên. Ví dụ trong tài liệu là `loan_propensity_au`.

Chạy thử, chỉ in quyết định, không commit:

```
uv run python -m deployer --dry-run
```

2 Đọc alias và lập kế hoạch (plan)

Với mỗi model, deployer đọc version đang @production và `storageUri` của artifact, so với manifest đang có trong Git, rồi ra một trong năm quyết định: **skip** (không có alias production), **unchanged** (manifest đã khớp), **blocked** (không thấy `<storageUri>/MLmodel` trên MinIO, tránh trỏ vào file chưa có), **create** (chưa có manifest) hoặc **update** (version khác).

Deployer chạy với --loop --interval 60 --verify-every 600; xem quyết định qua log:

```
kubectl -n mlops logs -f deploy/deployer
```

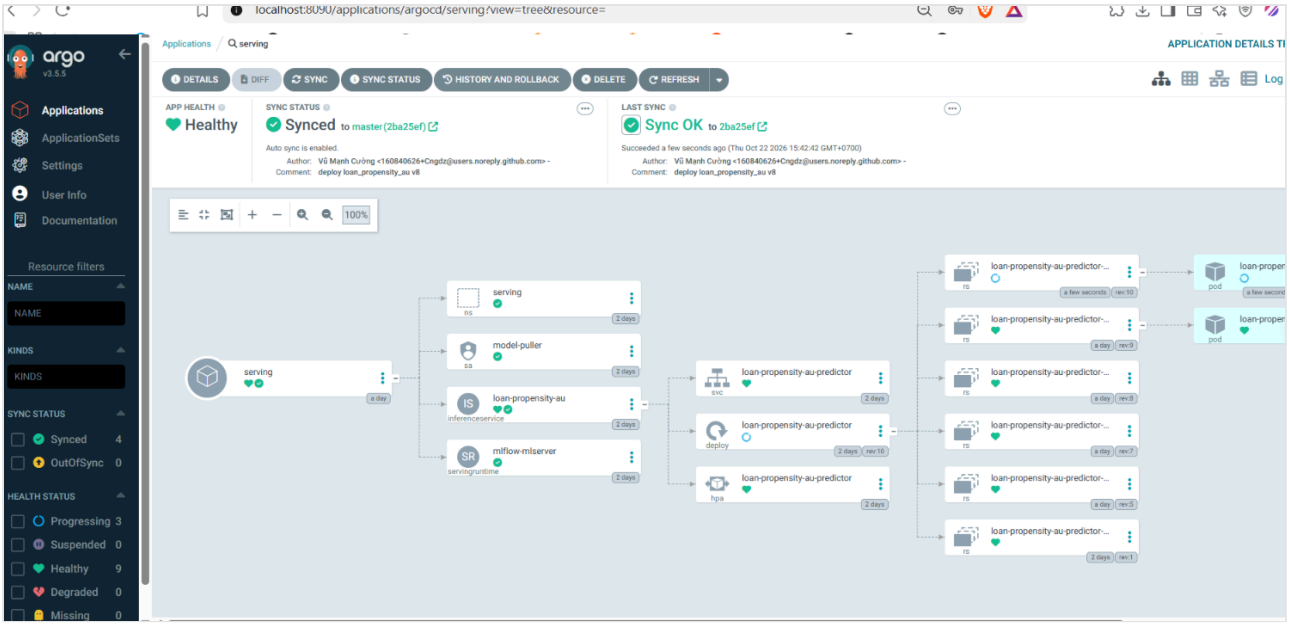
3 Commit manifest lên GitHub

Với create hoặc update, deployer commit file `serving/k8s/30-isvc-<model>.yaml` qua GitHub API với message `deploy <model> v<version>`. Manifest chứa annotation `mlops/model-version` và `storageUri` của version. Đổi annotation làm đổi spec của pod nên KServe tạo pod mới.

4 ArgoCD đồng bộ

Application `serving` bật auto-sync, prune, self-heal và ServerSideApply. ArgoCD hỏi Git theo chu kỳ (khoảng 60 giây cộng ngẫu nhiên), muốn nhanh thì bấm **Refresh**. Nếu ai sửa tay tài nguyên trong cụm, self-heal hoàn nguyên sau khoảng 2 giây.

```
kubectl -n argocd get applications
kubectl -n argocd get application serving -o jsonpath='{.status.sync.status} {.status.health.status}'
```



Hình 6.2: ArgoCD: Application serving ở trạng thái Healthy và Synced, cây tài nguyên InferenceService / Deployment / Pod

5 KServe tạo pod mới

KServe (RawDeployment) tạo Deployment. Container `storage-initializer` tải model từ MinIO theo `storageUri`, rồi MLServer (runtime `mlflow-mlserver`) nạp model. Pod cũ chỉ bị gỡ khi pod mới sẵn sàng nên không gián đoạn. Đo thực tế: commit → ArgoCD đồng bộ khoảng 23 giây → pod Ready thêm khoảng 17 giây.

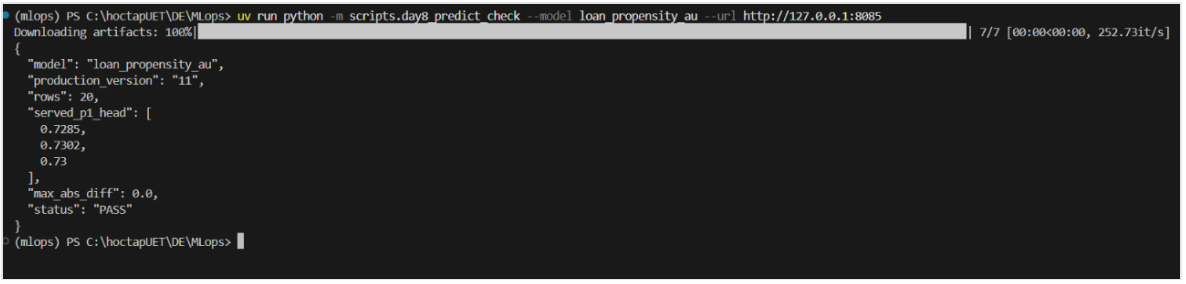
```
kubectl -n serving get isvc
kubectl -n serving get pods
```

6 Kiểm tra endpoint

Mở cổng tới dịch vụ rồi chạy script kiểm tra. Script gọi endpoint và so điểm với model nạp trực tiếp từ MLflow; kết quả PASS khi độ lệch tối đa `max_abs_diff` bằng 0.

Mở cổng rồi chạy kiểm tra:

```
kubectl -n serving port-forward svc/loan-propensity-au-predictor 8085:80
uv run python -m scripts.day8_predict_check --model loan_propensity_au --url http://127.0.0.1:8085
```



Hình 6.3: Kết quả scripts.day8_predict_check: version 11, status PASS, max_abs_diff 0.0

7 Vòng kiểm tra định kỳ (verify)

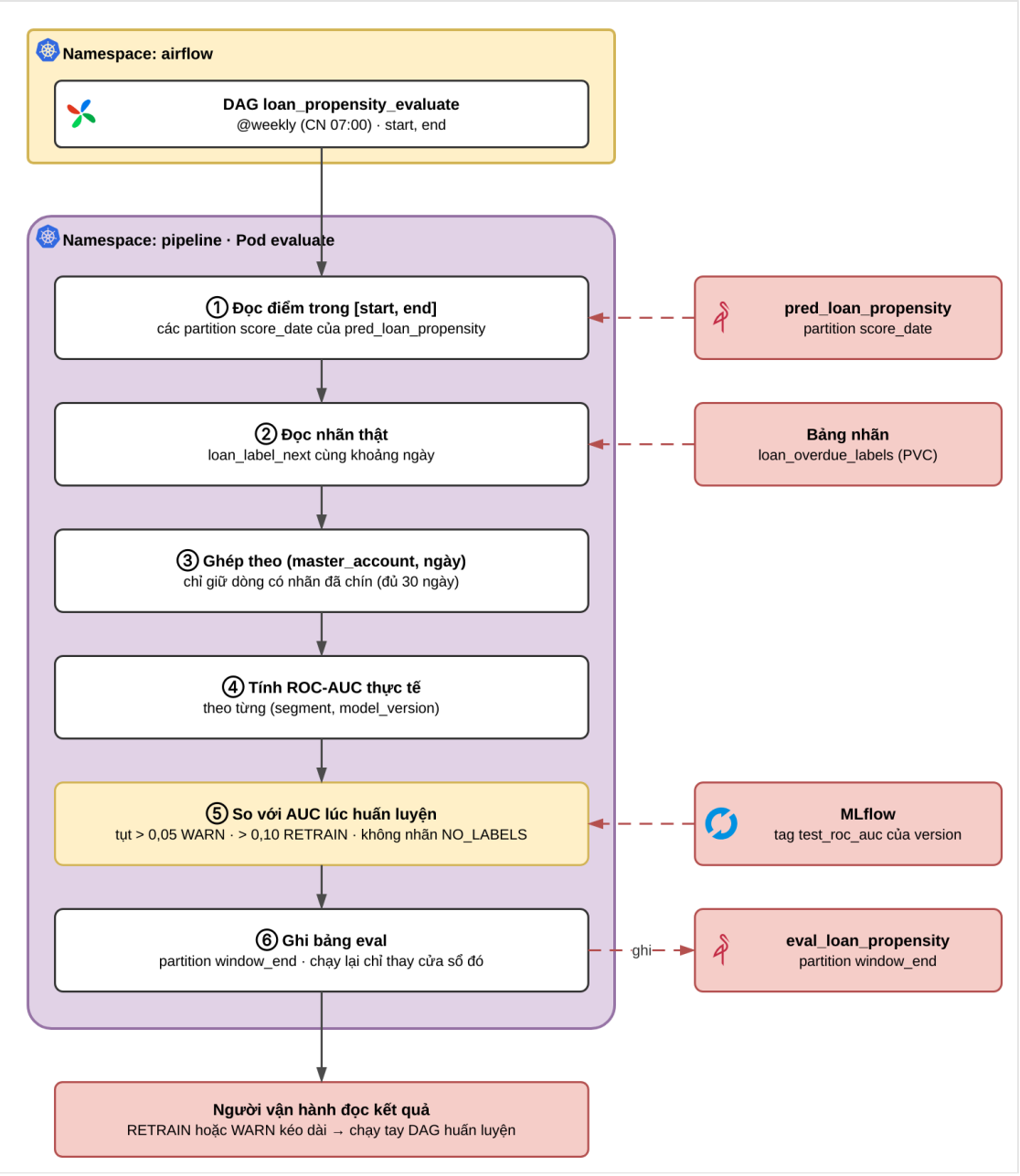
Cứ 600 giây deployer chạy `kubectl get isvc` và so annotation `mlops/model-version` với alias @production, trả về: **ok**, **lagging** (cụm chưa theo kịp), **not_ready** (pod chưa sẵn sàng), **missing** (chưa có InferenceService). Vòng này chỉ báo cáo, không tự sửa.

> **Lưu ý phạm vi.** Dự đoán batch (luồng 3) nạp model trực tiếp từ MLflow, nên việc triển khai KServe không làm thay đổi điểm batch. KServe là lớp dự đoán trực tuyến đã sẵn sàng cho các hệ thống cần gọi lẻ từng khách hàng; việc tích hợp phía gọi là hướng phát triển (mục 6.2).

7. Luồng 5: Đánh giá thực tế và huấn luyện lại

7.1 Tổng quan

Luồng này trả lời câu hỏi “model production đang tốt đến đâu so với lúc huấn luyện?”. DAG `loan_propensity_evaluate` chạy `@weekly` (Chủ Nhật 00:00 UTC = 07:00 giờ VN) với tham số `start` (mặc định 2025-12-25) và `end` (mặc định 2025-12-31). Vì nhãn cần 30 ngày để “chín”, cửa sổ đánh giá luôn lùi về quá khứ so với ngày chấm.



Hình 7.1: Luồng đánh giá hiệu quả thực tế

7.2 Từng bước

1 Đọc điểm dự đoán trong cửa sổ

Đọc các partition `score_date` nằm trong [start, end] của `pred_loan_propensity`, giữ cột model_version để biết mỗi dòng do version nào chấm.

```
kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_evaluate --conf
'{"start":"2025-12-25","end":"2025-12-31"}'
```

2 Ghép với nhãn thật

Đọc `loan_label_next` từ `loan_overdue_labels.parquet` và ghép theo (master_account, ngày). Chỉ giữ các dòng đã có nhãn. Ví dụ cửa sổ 25–31/12/2025 ghép được 33.117 dòng (au 13.097, lu 12.860, nu 7.160).

3 Tính ROC-AUC thực tế

Tính ROC-AUC theo từng cặp (segment, model_version) rồi so với tag `test_roc_auc` của chính version đó. Ví dụ với version 11: au 0,850 so với 0,829; lu 0,848 so với 0,817; nu 0,908 so với 0,889. AUC thực tế không thấp hơn lúc huấn luyện nên mức tụt âm.

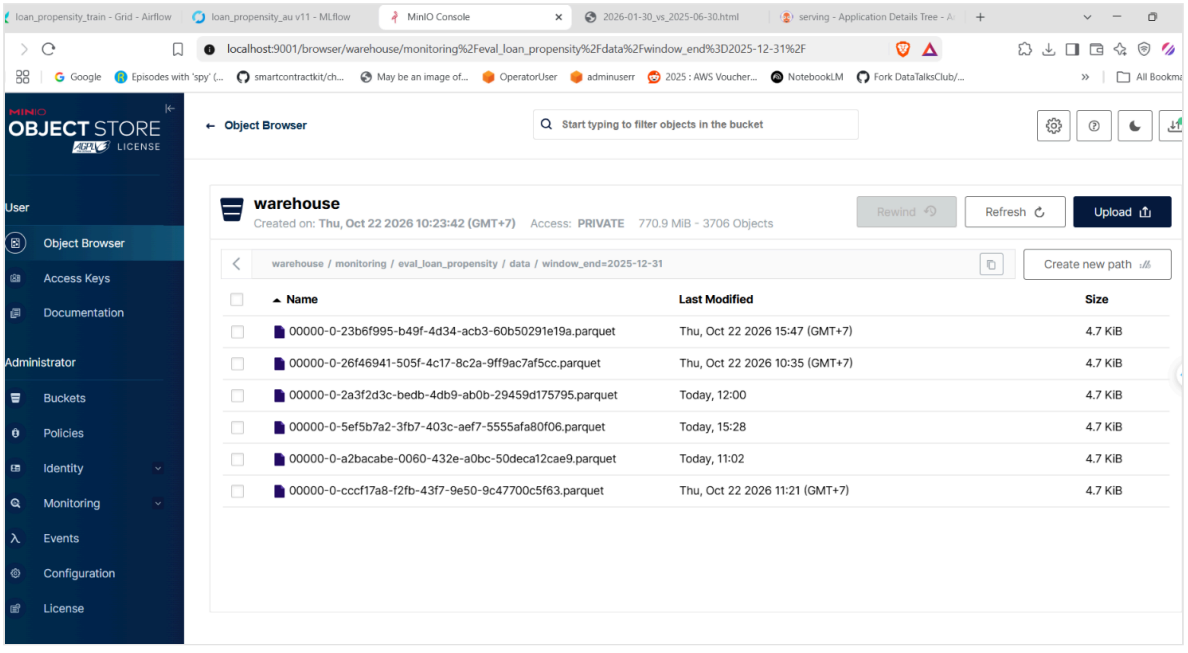
4 Phân loại mức cảnh báo

Mức tụt = AUC lúc huấn luyện − AUC thực tế. Tụt hơn 0,05: **WARN**; tụt hơn 0,10: **RETRAIN**; không có nhãn trong cửa sổ: **NO_LABELS**; còn lại **OK**. Cả ba segment trong ví dụ đều OK.

5 Ghi bảng eval

Ghi `monitoring.eval_loan_propensity`, partition `window_end`; chạy lại cùng (start, end) chỉ thay kết quả của cửa sổ đó. Mỗi lần ghi đè Iceberg có thể để lại file cũ chưa dọn (như ảnh dưới, một partition có sáu file ở các thời điểm khác nhau); dữ liệu đúng vẫn là snapshot mới nhất.

```
uv run python docs/screenshots/helpers/show_monitoring.py
```

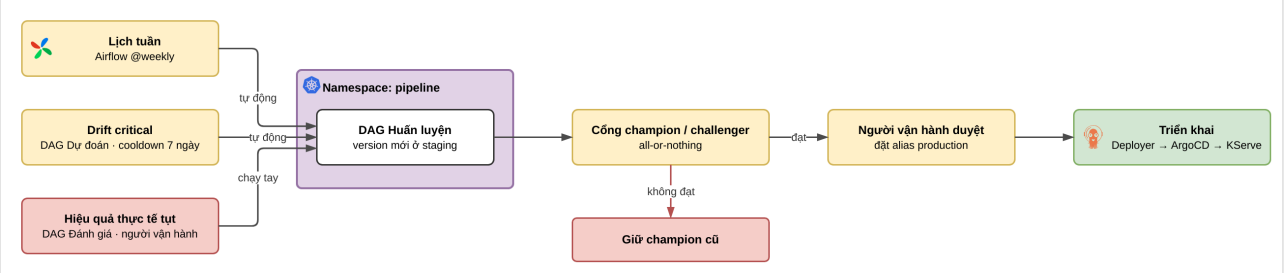


Hình 7.2: MinIO: partition window_end=2025-12-31 của bảng eval_loan_propensity

> **Lưu ý khi đọc kết quả.** Dữ liệu tháng 10–12/2025 cũng là tập test dùng cho cổng champion, nên đây chưa phải kiểm tra trên dữ liệu độc lập hoàn toàn. Đó cũng là lý do AUC của một tuần có thể cao hơn AUC của cả tập test (mức tụt âm). Khi hệ thống chạy liên tục hằng ngày, cửa sổ đánh giá sẽ là dữ liệu mới phát sinh sau thời điểm huấn luyện.

Hai bài toán còn lại dùng DAG `overdue_risk_evaluate` và `campaign_response_evaluate` với cùng ngưỡng WARN / RETRAIN; nhãn lần lượt là `overdue_label_next_30d` (chín sau 30 ngày) và `campaign_response_label_14d` (chín sau 14 ngày), ghi vào bảng `monitoring.eval_<usecase>`.

7.3 Vòng huấn luyện lại



Hình 7.3: Ba đường dẫn tới huấn luyện lại; mọi đường đều qua cổng champion và người vận hành duyệt

Bảng 7.1: Điều kiện kích hoạt huấn luyện lại (mọi đường đều kết thúc ở version staging, cần cổng champion và người vận hành duyệt mới có production)

Tín hiệu	Từ đâu	Hành động
Lịch tuần	Airflow @weekly	DAG huấn luyện tự chạy
Drift critical + có nhãn mới + hết cooldown	DAG dự đoán (retrain_ check)	Tự kích hoạt DAG huấn luyện
Đánh giá WARN kéo dài hoặc RETRAIN	DAG đánh giá	Người vận hành xem bảng eval và chạy tay DAG huấn luyện

8. Tổng kết và lệnh thường dùng

8.1 Tổng kết

Bảng 8.1: Tổng kết các luồng

Luồng	Chạy khi nào	Kết quả chính
1. Gold → Feature Store	Hằng ngày 06:00 giờ VN	Partition `report_date` mới ở 6 bảng fs_*, ngày được đánh dấu hoàn tất
2. Huấn luyện	Chủ Nhật 07:00 hoặc khi drift critical	Version mới của từng model ở staging (6 model cho 3 bài toán); cổng champion; người vận hành duyệt production
3. Dự đoán và drift	Hằng ngày 07:00	Điểm dự đoán và kết quả drift theo ngày; quyết định huấn luyện lại
4. Triển khai	Khi alias production đổi	InferenceService phục vụ đúng version production, nền tảng cho dự đoán trực tuyến
5. Đánh giá	Chủ Nhật 07:00	AUC thực tế so với lúc huấn luyện, cảnh báo WARN / RETRAIN

- Mọi thay đổi model đều qua cổng champion / challenger và sự duyệt của người vận hành.
- Mọi thay đổi trên cụm phục vụ đều có commit Git tương ứng và có thể quay lại version trước bằng cách đặt lại alias.
- Các bảng giám sát ghi theo partition ngày nên chạy lại một ngày không ảnh hưởng ngày khác.

8.2 Lệnh thường dùng

Bảng 8.2: Lệnh thường dùng

Việc	Lệnh
Mở MLflow UI	`kubectl -n mlops port-forward svc/mlflow 5050:80`
Danh sách DAG	`kubectl -n airflow exec deploy/airflow -- airflow dags list`
Chạy tay DAG huấn luyện	`kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_train`
Chạy tay DAG dự đoán	`kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_predict --conf '{"score_date":"2026-01-30"}'`
Chạy tay DAG đánh giá	`kubectl -n airflow exec deploy/airflow -- airflow dags trigger loan_propensity_evaluate`
Xem kế hoạch chuyển production	`uv run python -m training.promote --usecase loan_propensity`
Duyệt production	`uv run python -m training.promote --usecase loan_propensity --approve`
Log deployer	`kubectl -n mlops logs -f deploy/deployer`
Xem InferenceService	`kubectl -n serving get isvc`
Kiểm tra endpoint	`uv run python -m scripts.day8_predict_check --model loan_propensity_au --url http://127.0.0.1:8085`
Xem số dòng bảng giám sát	`uv run python docs/screenshots/helpers/show_monitoring.py`

8.3 Hạn chế và hướng phát triển

- **Chỉ số nghiệp vụ**: thêm precision và lift ở top-k% theo năng lực liên hệ của bộ phận kinh doanh.
- **Cảnh báo**: hiện xem trên Airflow, bảng eval và tag MLflow; bổ sung gửi Slack / email.
- **Giám sát drift**: loại các cột tăng tự nhiên theo thời gian (như `account_age_days`) khỏi phép đếm, ngưỡng riêng theo cột.
- **Dự đoán trực tuyến**: gắn API gateway và Feast / Redis vào InferenceService KServe đã có; đưa thêm model của overdue_risk, campaign_response lên KServe.